CornerNet: Detecting Objects as Paired Keypoints

Hei Law · **Jia Deng**

Abstract We propose CornerNet, a new approach to object detection where we detect an object bounding box as a pair of keypoints, the top-left corner and the bottom-right corner, using a single convolution neural network. By detecting objects as paired keypoints, we eliminate the need for designing a set of anchor boxes commonly used in prior single-stage detectors. In addition to our novel formulation, we introduce corner pooling, a new type of pooling layer that helps the network better localize corners. Experiments show that CornerNet achieves a 42.2% AP on MS COCO, outperforming all existing one-stage detectors.

Keywords Object Detection

1 Introduction

Object detectors based on convolutional neural networks (ConvNets) (Krizhevsky et al., 2012; Simonyan and Zisserman, 2014; He et al., 2016) have achieved state-of-the-art results on various challenging benchmarks (Lin et al., 2014; Deng et al., 2009; Everingham et al., 2015). A common component of state-of-the-art approaches is anchor boxes (Ren et al., 2015; Liu et al., 2016), which are boxes of various sizes and aspect ratios that serve as detection candidates. Anchor boxes are extensively used in one-stage detectors (Liu et al., 2016; Fu et al., 2017; Redmon and Farhadi, 2016; Lin et al., 2017), which can achieve results highly competitive with two-stage detectors (Ren et al., 2015; Girshick et al., 2014; Girshick, 2015; He et al., 2017) while being more efficient. One-stage detectors place anchor boxes densely over an image and generate final box predictions by scoring anchor boxes and refining their coordinates through regression.

But the use of anchor boxes has two drawbacks. First, we typically need a very large set of anchor boxes, e.g. more than 40k in DSSD (Fu et al., 2017) and more than 100k in RetinaNet (Lin et al., 2017). This is because the detector is trained to classify whether each anchor box sufficiently overlaps with a ground truth box, and a large number of anchor boxes is needed to ensure sufficient overlap with most ground truth boxes. As a result, only a tiny fraction of anchor boxes will overlap with ground truth; this creates a huge imbalance between positive and negative anchor boxes and slows down training (Lin et al., 2017).

Second, the use of anchor boxes introduces many hyperparameters and design choices. These include how many boxes, what sizes, and what aspect ratios. Such choices have largely been made via ad-hoc heuristics, and can become even more complicated when combined with multiscale architectures where a single network makes separate predictions at multiple resolutions, with each scale using different features and its own set of anchor boxes (Liu et al., 2016; Fu et al., 2017; Lin et al., 2017).

In this paper we introduce CornerNet, a new one-stage approach to object detection that does away with anchor boxes. We detect an object as a pair of keypoints—the top-left corner and bottom-right corner of the bounding box. We use a single convolutional network to predict a heatmap for the top-left corners of all instances of the same object category, a heatmap for all bottom-right corners, and an embedding vector for each detected corner. The embeddings serve to group a pair of corners that belong to the same object—the network is trained to predict similar embeddings for them. Our ap-

H. Law
Princeton University, Princeton, NJ, USA
E-mail: heilaw@cs.princeton.edu

J. Deng
Princeton Universtiy, Princeton, NJ, USA

CornerNet: Detecting Objects as Paired Keypoints

Hei Law · Jia Deng

Abstract 我们提出了CornerNet，一种新的目标检测方法，该方法通过单个卷积神经网络将目标边界框检测为一对关键点——左上角和右下角。通过将目标检测为成对的关键点，我们消除了先前单阶段检测器中常用的锚框设计需求。除了我们新颖的公式化方法外，我们还引入了角点池化，这是一种新型的池化层，有助于网络更好地定位角点。实验表明，CornerNet在MS COCO数据集上实现了42.2%的AP，优于所有现有的一阶段检测器。

Keywords 目标检测

1 Introduction

基于卷积神经网络（ConvNets）（Krizhevsky等人，2012；Simonyan与Zisserman，2014；He等人，2016）的目标检测器已在各类具有挑战性的基准测试（Lin等人，2014；Deng等人，2009；Everingham等人，2015）中取得了最先进的成果。当前主流方法的一个共同组件是锚框（Ren等人，2015；Liu等人，2016），即多种尺寸和宽高比的候选检测框。锚框被广泛应用于单阶段检测器（Liu等人，2016；Fu等人，2017；Redmon与Farhadi，2016；Lin等人，2017），这类检测器能够取得与两阶段检测器（Ren等人，2015；Girshick等人，2014；Girshick，2015；He等人，2017）高度竞争的结果，同时更高效。单阶段检测器在图像上密集放置锚框，并通过评分锚框和通过回归精调其坐标来生成最终的框预测。

但锚框的使用存在两个缺点。首先，我们通常需要非常庞大的锚框集合，例如DSSD（Fu等人，2017）中超过4万个，RetinaNet（Lin等人，2017）中超过10万个。这是因为检测器被训练用于判断每个锚框是否与真实标注框有足够重叠，而需要大量锚框才能确保与大多数真实标注框充分重叠。结果，只有极小比例的锚框会与真实标注框重叠；这导致正负锚框之间出现严重不平衡，并减缓训练速度（Lin等人，2017）。

其次，锚框的使用引入了许多超参数和设计选择。这包括框的数量、大小以及宽高比。这些选择大多通过临时启发式方法确定，并且在与多尺度架构结合时会变得更加复杂，因为单个网络会在多个分辨率下进行独立预测，每个尺度使用不同的特征及其自身的一组锚框（Liu et al., 2016; Fu et al., 2017; Lin et al., 2017）。

本文提出CornerNet，一种无需锚框的单阶段目标检测新方法。我们将目标检测视为一对关键点（边界框的左上角与右下角）的定位问题。通过单一卷积网络预测同一物体类别所有实例的左上角热力图、右下角热力图，并为每个检测到的角点生成嵌入向量。嵌入向量用于将属于同一物体的角点进行匹配——网络通过训练学会为同一物体的角点预测相似的嵌入向量。

H. Law
Princeton University, Princeton, NJ, USA
E-mail: heilaw@cs.princeton.edu

J. Deng
Princeton Universtiy, Princeton, NJ, USA

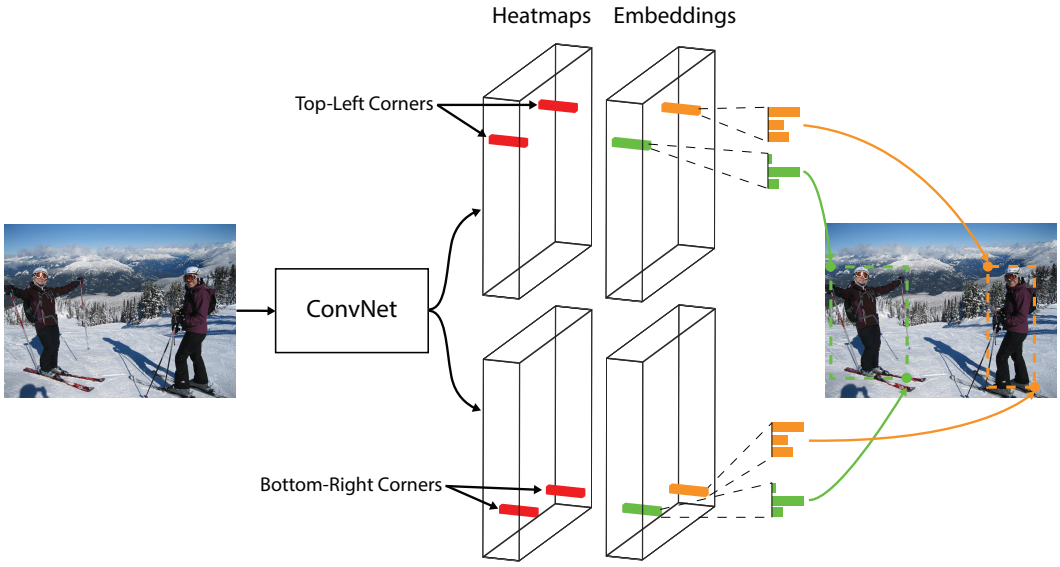


Fig. 1 We detect an object as a pair of bounding box corners grouped together. A convolutional network outputs a heatmap for all top-left corners, a heatmap for all bottom-right corners, and an embedding vector for each detected corner. The network is trained to predict similar embeddings for corners that belong to the same object.

proach greatly simplifies the output of the network and eliminates the need for designing anchor boxes. Our approach is inspired by the associative embedding method proposed by Newell et al. (2017), who detect and group keypoints in the context of multiperson human-pose estimation. Fig. 1 illustrates the overall pipeline of our approach.

Another novel component of CornerNet is *corner pooling*, a new type of pooling layer that helps a convolutional network better localize corners of bounding boxes. A corner of a bounding box is often outside the object—consider the case of a circle as well as the examples in Fig. 2. In such cases a corner cannot be localized based on local evidence. Instead, to determine whether there is a top-left corner at a pixel location, we need to look horizontally towards the right for the topmost boundary of the object, and look vertically towards the bottom for the leftmost boundary. This motivates our corner pooling layer: it takes in two feature maps; at each pixel location it max-pools all feature vectors to the right from the first feature map, max-pools all feature vectors directly below from the second feature map, and then adds the two pooled results together. An example is shown in Fig. 3.

We hypothesize two reasons why detecting corners would work better than bounding box centers or proposals. First, the center of a box can be harder to localize because it depends on all 4 sides of the object, whereas locating a corner depends on 2 sides and is thus easier, and even more so with corner pooling, which encodes some explicit prior knowledge about the definition of corners. Second, corners provide a more efficient way of densely discretizing the space of boxes: we just need $O(wh)$ corners to represent $O(w^2h^2)$ possible anchor boxes.

We demonstrate the effectiveness of CornerNet on MS COCO (Lin et al., 2014). CornerNet achieves a 42.2% AP, outperforming all existing one-stage detectors. In addition, through ablation studies we show that corner pooling is critical to the superior performance of CornerNet. Code is available at `https://github.com/princeton-vl/CornerNet`.

2 Related Works

2.1 Two-stage object detectors

Two-stage approach was first introduced and popularized by R-CNN (Girshick et al., 2014). Two-stage detectors generate a sparse set of regions of interest (RoIs) and classify each of them by a network. R-CNN generates RoIs using a low level vision algorithm (Uijlings et al., 2013; Zitnick and Dollár, 2014). Each region is then extracted from the image and processed by a ConvNet independently, which creates lots of redundant computations. Later, SPP (He et al., 2014) and Fast-RCNN (Girshick, 2015) improve R-CNN by designing a special pooling layer that pools each region from feature maps instead. However, both still rely on separate proposal algorithms and cannot be trained end-to-end. Faster-RCNN (Ren et al., 2015) does away low level proposal algorithms by introducing a region proposal network (RPN), which generates proposals from a set of

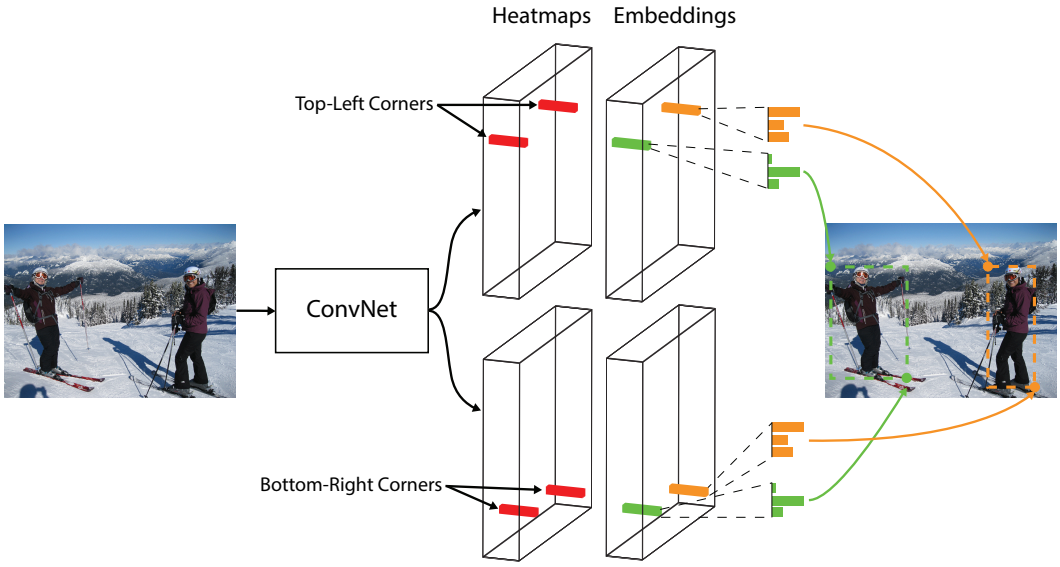


Fig. 1 我们将物体检测为一对组合在一起的边界框角点。一个卷积网络输出所有左上角的热图、所有右下角的热图，以及每个检测到的角点的嵌入向量。该网络经过训练，能为属于同一物体的角点预测相似的嵌入向量。

我们的方法极大地简化了网络的输出，并消除了设计锚框的需求。我们的方法受到Newell等人（2017）提出的关联嵌入方法的启发，该方法在多人体姿态估计的背景下检测并分组关键点。图1展示了我们方法的整体流程。

CornerNet的另一个新颖组件是*corner pooling*，这是一种新型池化层，可帮助卷积网络更精准地定位边界框的角点。边界框的角点通常位于物体外部——以圆形为例，图2中的示例也说明了这一点。在这种情况下，仅依靠局部信息无法定位角点。相反，要判断某个像素位置是否存在左上角点，我们需要水平向右寻找物体的最上边界，并垂直向下寻找物体的最左边界。这启发了我们设计角点池化层：该层接收两个特征图；在每个像素位置，它对第一个特征图中该像素右侧的所有特征向量进行最大池化，同时对第二个特征图中该像素正下方的所有特征向量进行最大池化，最后将两个池化结果相加。具体示例如图3所示。

我们假设检测角点比边界框中心或提议框效果更好的原因有两个。首先，框的中心可能更难定位，因为它依赖于物体的所有四条边，而定位一个角点只依赖于两条边，因此更容易，尤其是在使用角点池化时——它编码了关于角点定义的某些显式先验知识。其次，角点提供了更高效的方式。

密集离散化边界框空间的方法：我们只需要$O(wh)$个角点来表示$O(w^2h^2)$种可能的锚框。

我们在MS COCO数据集（Lin等人，2014）上验证了CornerNet的有效性。CornerNet取得了42.2%的平均精度（AP），超越了所有现有的一阶段检测器。此外，通过消融实验，我们证明了角点池化对CornerNet的优异性能至关重要。代码发布于`https://github.com/princeton-vl/CornerNet`。

2 Related Works

2.1 两阶段目标检测器

两阶段方法最初由R-CNN（Girshick等人，2014年）提出并推广。两阶段检测器首先生成一组稀疏的感兴趣区域（RoIs），再通过神经网络对每个区域进行分类。R-CNN采用低级视觉算法生成RoIs（Uijlings等人，2013年；Zitnick与Dollár，2014年）。每个区域从图像中提取后，需独立经过卷积网络处理，这产生了大量冗余计算。随后，SPP（He等人，2014年）与Fast-RCNN（Girshick，2015年）改进了R-CNN，设计了特殊的池化层，可直接从特征图中池化每个区域。但两者仍依赖独立的候选框生成算法，无法实现端到端训练。Faster-RCNN（Ren等人，2015年）通过引入区域提议网络（RPN）取代了低级候选框算法，该网络可直接从一组特征中生成候选框。

Fig. 2 Often there is no local evidence to determine the location of a bounding box corner. We address this issue by proposing a new type of pooling layer.

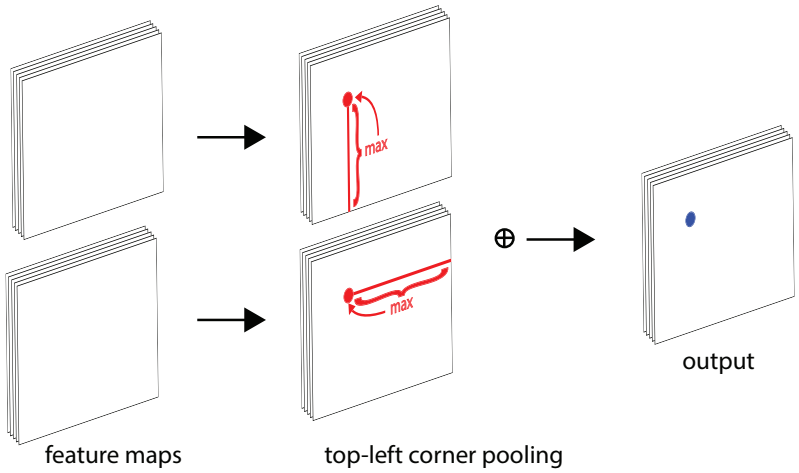


Fig. 3 Corner pooling: for each channel, we take the maximum values *(red dots)* in two directions *(red lines)*, each from a separate feature map, and add the two maximums together *(blue dot)*.

pre-determined candidate boxes, usually known as anchor boxes. This not only makes the detectors more efficient but also allows the detectors to be trained end-to-end. R-FCN (Dai et al., 2016) further improves the efficiency of Faster-RCNN by replacing the fully connected sub-detection network with a fully convolutional sub-detection network. Other works focus on incorporating sub-category information (Xiang et al., 2016), generating object proposals at multiple scales with more contextual information (Bell et al., 2016; Cai et al., 2016; Shrivastava et al., 2016; Lin et al., 2016), selecting better features (Zhai et al., 2017), improving speed (Li et al., 2017), cascade procedure (Cai and Vasconcelos, 2017) and better training procedure (Singh and Davis, 2017).

2.2 One-stage object detectors

On the other hand, YOLO (Redmon et al., 2016) and SSD (Liu et al., 2016) have popularized the one-stage approach, which removes the RoI pooling step and detects objects in a single network. One-stage detectors are usually more computationally efficient than two-stage detectors while maintaining competitive performance on different challenging benchmarks.

SSD places anchor boxes densely over feature maps from multiple scales, directly classifies and refines each anchor box. YOLO predicts bounding box coordinates directly from an image, and is later improved in YOLO9000 (Redmon and Farhadi, 2016) by switching to anchor boxes. DSSD (Fu et al., 2017) and RON (Kong et al., 2017) adopt networks similar to the hourglass network (Newell et al., 2016), enabling them to combine low-level and high-level features via skip connections to predict bounding boxes more accurately. However, these one-stage detectors are still outperformed by the two-stage detectors until the introduction of RetinaNet (Lin et al., 2017). In (Lin et al., 2017), the authors suggest that the dense anchor boxes create a huge imbalance between positive and negative anchor boxes during training. This imbalance causes the training to be inefficient and hence the performance to be suboptimal. They propose a new loss, Focal Loss, to dynamically adjust the weights of each anchor box and show that their one-stage detector can outperform the two-stage detectors. RefineDet (Zhang et al., 2017) proposes to filter the an-

Fig. 2 通常没有局部证据来确定边界框角点的位置。我们通过提出一种新型的池化层来解决这个问题。

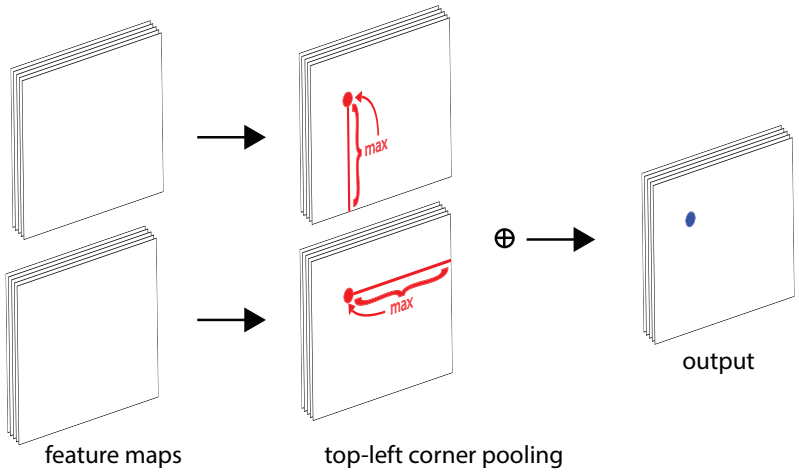


Fig. 3 角点池化：对于每个通道，我们从两个方向*(red lines)*分别取两个特征图中的最大值*(red dots)*，然后将这两个最大值相加*(blue dot)*。

预先确定的候选框，通常称为锚框。这不仅使检测器更加高效，还允许检测器进行端到端的训练。R-FCN（Dai等人，2016）通过将全连接子检测网络替换为全卷积子检测网络，进一步提升了Faster-RCNN的效率。其他研究则侧重于整合子类别信息（Xiang等人，2016）、利用更多上下文信息生成多尺度目标提议（Bell等人，2016；Cai等人，2016；Shrivastava等人，2016；Lin等人，2016）、选择更好的特征（Zhai等人，2017）、提升速度（Li等人，2017）、级联流程（Cai和Vasconcelos，2017）以及改进训练流程（Singh和Davis，2017）。

2.2 单阶段目标检测器

另一方面，YOLO（Redmon等人，2016年）和SSD（Liu等人，2016年）推广了单阶段方法，该方法省去了RoI池化步骤，直接在单一网络中检测物体。单阶段检测器通常比两阶段检测器计算效率更高。

阶段检测器，同时在各种具有挑战性的基准测试中保持有竞争力的性能。

SSD在多个尺度的特征图上密集放置锚框，直接对每个锚框进行分类和优化。YOLO直接从图像预测边界框坐标，后来在YOLO9000（Redmon和Farhadi，2016）中通过改用锚框进行了改进。DSSD（Fu等人，2017）和RON（Kong等人，2017）采用了类似沙漏网络（Newell等人，2016）的结构，通过跳跃连接结合低级和高级特征，以更准确地预测边界框。然而，在RetinaNet（Lin等人，2017）提出之前，这些单阶段检测器的性能仍不及两阶段检测器。在（Lin等人，2017）中，作者指出密集的锚框在训练期间会导致正负锚框之间的严重不平衡。这种不平衡使得训练效率低下，从而导致性能欠佳。他们提出了一种新的损失函数——Focal Loss，以动态调整每个锚框的权重，并证明其单阶段检测器能够超越两阶段检测器。RefineDet（Zhang等人，2017）提出过滤锚

chor boxes to reduce the number of negative boxes, and to coarsely adjust the anchor boxes.

DeNet (Tychsen-Smith and Petersson, 2017a) is a two-stage detector which generates RoIs without using anchor boxes. It first determines how likely each location belongs to either the top-left, top-right, bottom-left or bottom-right corner of a bounding box. It then generates RoIs by enumerating all possible corner combinations, and follows the standard two-stage approach to classify each RoI. Our approach is very different from DeNet. First, DeNet does not identify if two corners are from the same objects and relies on a sub-detection network to reject poor RoIs. In contrast, our approach is a one-stage approach which detects and groups the corners using a single ConvNet. Second, DeNet selects features at manually determined locations relative to a region for classification, while our approach does not require any feature selection step. Third, we introduce corner pooling, a novel type of layer to enhance corner detection.

Point Linking Network (PLN) (Wang et al., 2017) is an one-stage detector without anchor boxes. It first predicts the locations of the four corners and the center of a bounding box. Then, at each corner location, it predicts how likely each pixel location in the image is the center. Similarly, at the center location, it predicts how likely each pixel location belongs to either the top-left, top-right, bottom-left or bottom-right corner. It combines the predictions from each corner and center pair to generate a bounding box. Finally, it merges the four bounding boxes to give a bounding box. CornerNet is very different from PLN. First, CornerNet groups the corners by predicting embedding vectors, while PLN groups the corner and center by predicting pixel locations. Second, CornerNet uses corner pooling to better localize the corners.

Our approach is inspired by Newell et al. (2017) on Associative Embedding in the context of multi-person pose estimation. Newell et al. propose an approach that detects and groups human joints in a single network. In their approach each detected human joint has an embedding vector. The joints are grouped based on the distances between their embeddings. To the best of our knowledge, we are the first to formulate the task of object detection as a task of detecting and grouping corners with embeddings. Another novelty of ours is the corner pooling layers that help better localize the corners. We also significantly modify the hourglass architecture and add our novel variant of focal loss (Lin et al., 2017) to help better train the network.

3 CornerNet

3.1 Overview

In CornerNet, we detect an object as a pair of keypoints—the top-left corner and bottom-right corner of the bounding box. A convolutional network predicts two sets of heatmaps to represent the locations of corners of different object categories, one set for the top-left corners and the other for the bottom-right corners. The network also predicts an embedding vector for each detected corner (Newell et al., 2017) such that the distance between the embeddings of two corners from the same object is small. To produce tighter bounding boxes, the network also predicts offsets to slightly adjust the locations of the corners. With the predicted heatmaps, embeddings and offsets, we apply a simple post-processing algorithm to obtain the final bounding boxes.

Fig. 4 provides an overview of CornerNet. We use the hourglass network (Newell et al., 2016) as the backbone network of CornerNet. The hourglass network is followed by two prediction modules. One module is for the top-left corners, while the other one is for the bottom-right corners. Each module has its own corner pooling module to pool features from the hourglass network before predicting the heatmaps, embeddings and offsets. Unlike many other object detectors, we do not use features from different scales to detect objects of different sizes. We only apply both modules to the output of the hourglass network.

3.2 Detecting Corners

We predict two sets of heatmaps, one for top-left corners and one for bottom-right corners. Each set of heatmaps has C channels, where C is the number of categories, and is of size $H \times W$. There is no background channel. Each channel is a binary mask indicating the locations of the corners for a class.

For each corner, there is one ground-truth positive location, and all other locations are negative. During training, instead of equally penalizing negative locations, we reduce the penalty given to negative locations within a radius of the positive location. This is because a pair of false corner detections, if they are close to their respective ground truth locations, can still produce a box that sufficiently overlaps the ground-truth box (Fig. 5). We determine the radius by the size of an object by ensuring that a pair of points within the radius would generate a bounding box with at least t IoU with the ground-truth annotation (we set t to 0.3 in all experiments). Given the radius, the amount of penalty reduction is given by an unnormalized 2D Gaussian,

chor框以减少负样本框的数量，并粗略调整锚框。

DeNet（Tychsen-Smith和Petersson，2017a）是一种无需使用锚框即可生成感兴趣区域的两阶段检测器。它首先判断每个位置属于边界框左上角、右上角、左下角或右下角的可能性，随后通过枚举所有可能的角点组合生成感兴趣区域，并遵循标准的两阶段方法对每个感兴趣区域进行分类。我们的方法与DeNet存在显著差异。首先，DeNet不判断两个角点是否属于同一物体，而是依赖子检测网络剔除低质量的感兴趣区域；相比之下，我们的方法采用单阶段检测，通过单个卷积网络同时完成角点检测与分组。其次，DeNet需根据区域相对位置手动选择特征进行分类，而我们的方法无需任何特征选择步骤。第三，我们引入了角点池化这一新型网络层来增强角点检测能力。

点连接网络（PLN）（Wang等人，2017）是一种无需锚框的单阶段检测器。它首先预测边界框的四个角点和中心点的位置。接着，在每个角点位置，网络预测图像中每个像素位置作为中心点的可能性；类似地，在中心点位置，网络预测每个像素位置属于左上、右上、左下或右下角点的可能性。通过结合每个角点与中心点的预测结果，网络生成边界框，最终合并四个边界框以输出最终检测框。CornerNet与PLN存在显著差异：首先，CornerNet通过预测嵌入向量来分组角点，而PLN通过预测像素位置来关联角点与中心点；其次，CornerNet采用角点池化技术以更精准地定位角点。

我们的方法受到Newell等人（2017）在多人体姿态估计中提出的关联嵌入（Associative Embedding）的启发。Newell等人提出了一种在单一网络中检测并分组人体关节点的方法。在他们的方法中，每个检测到的人体关节点都拥有一个嵌入向量，关节点的分组依据其嵌入向量之间的距离进行。据我们所知，我们是首个将目标检测任务形式化为通过嵌入向量检测并分组角点的研究。我们的另一创新是引入了角点池化层，以提升角点的定位精度。此外，我们大幅改进了沙漏网络结构，并加入了我们基于焦点损失（Lin等人，2017）的新颖变体，以优化网络训练效果。

3 CornerNet

3.1 概述

在CornerNet中，我们将物体检测为一对关键点——边界框的左上角点和右下角点。一个卷积网络预测两组热图来表示不同物体类别的角点位置，一组用于左上角，另一组用于右下角。该网络还为每个检测到的角点预测一个嵌入向量（Newell等人，2017），使得来自同一物体的两个角点的嵌入向量之间的距离较小。为了生成更紧密的边界框，网络还预测偏移量以微调角点的位置。利用预测的热图、嵌入向量和偏移量，我们应用一个简单的后处理算法来获得最终的边界框。

图4提供了CornerNet的概览。我们采用沙漏网络（Newell等人，2016）作为CornerNet的骨干网络。沙漏网络后接两个预测模块：一个模块用于检测左上角点，另一个模块用于检测右下角点。每个模块均配备独立的角点池化模块，用于在预测热力图、嵌入向量和偏移量前从沙漏网络汇聚特征。与许多其他目标检测器不同，我们未使用多尺度特征来检测不同尺寸的目标，仅将两个模块统一应用于沙漏网络的输出。

3.2 角点检测

我们预测两组热图，一组用于左上角，一组用于右下角。每组热图有C个通道，其中C是类别数量，尺寸为$H \times W$。没有背景通道。每个通道都是一个二进制掩码，指示某个类别的角点位置。

对于每个角点，存在一个真实正样本位置，其余所有位置均为负样本。在训练过程中，我们并非对所有负样本位置施加同等惩罚，而是降低对正样本位置半径范围内负样本的惩罚力度。这是因为一对错误的角点检测，只要它们各自接近对应的真实位置，仍可能生成与真实标注框有足够重叠的检测框（图5）。我们根据目标尺寸确定半径范围，确保半径内的一对点生成的边界框与真实标注框至少具有t的交并比（实验中我们设定t为0.3）。基于该半径，惩罚力度的衰减值由一个非标准化的二维高斯函数给出，

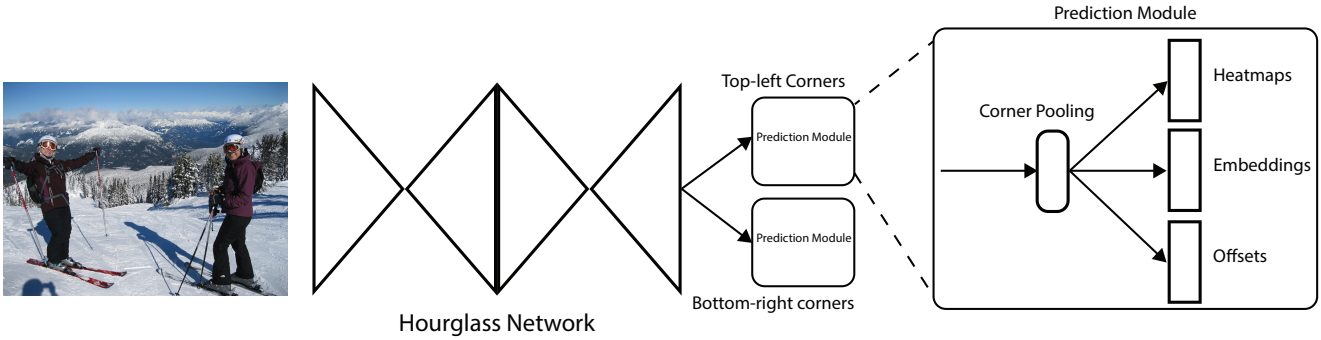


Fig. 4 Overview of CornerNet. The backbone network is followed by two prediction modules, one for the top-left corners and the other for the bottom-right corners. Using the predictions from both modules, we locate and group the corners.

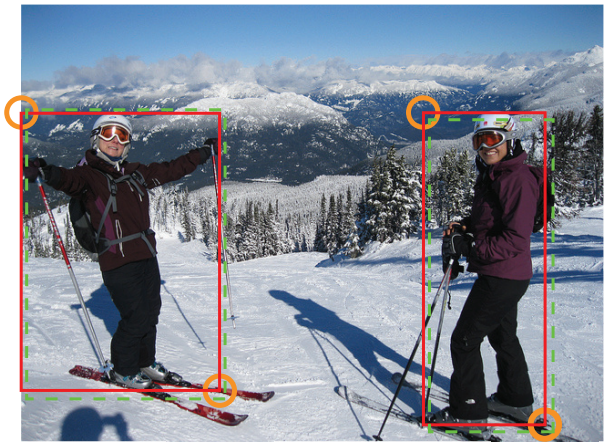


Fig. 5 "Ground-truth" heatmaps for training. Boxes *(green dotted rectangles)* whose corners are within the radii of the positive locations *(orange circles)* still have large overlaps with the ground-truth annotations *(red solid rectangles)*.

$e^{-\frac{x^2+y^2}{2\sigma^2}}$, whose center is at the positive location and whose σ is 1/3 of the radius.

Let p_{cij} be the score at location (i, j) for class c in the predicted heatmaps, and let y_{cij} be the "ground-truth" heatmap augmented with the unnormalized Gaussians. We design a variant of focal loss (Lin et al., 2017):

$$L_{det} = \frac{-1}{N} \sum_{c=1}^{C} \sum_{i=1}^{H} \sum_{j=1}^{W} \begin{cases} (1-p_{cij})^{\alpha} \log(p_{cij}) & \text{if } y_{cij} = 1 \\ (1-y_{cij})^{\beta} (p_{cij})^{\alpha} \log(1-p_{cij}) & \text{otherwise} \end{cases} \quad (1)$$

where N is the number of objects in an image, and α and β are the hyper-parameters which control the contribution of each point (we set α to 2 and β to 4 in all experiments). With the Gaussian bumps encoded in y_{cij}, the $(1-y_{cij})$ term reduces the penalty around the ground truth locations.

Many networks (He et al., 2016; Newell et al., 2016) involve downsampling layers to gather global information and to reduce memory usage. When they are applied to an image fully convolutionally, the size of the output is usually smaller than the image. Hence, a location (x, y) in the image is mapped to the location $\left(\lfloor \frac{x}{n} \rfloor, \lfloor \frac{y}{n} \rfloor\right)$ in the heatmaps, where n is the downsampling factor. When we remap the locations from the heatmaps to the input image, some precision may be lost, which can greatly affect the IoU of small bounding boxes with their ground truths. To address this issue we predict location offsets to slightly adjust the corner locations before remapping them to the input resolution.

$$\boldsymbol{o}_k = \left(\frac{x_k}{n} - \left\lfloor \frac{x_k}{n} \right\rfloor, \frac{y_k}{n} - \left\lfloor \frac{y_k}{n} \right\rfloor\right) \quad (2)$$

where $\boldsymbol{o}_k$ is the offset, x_k and y_k are the x and y coordinate for corner k. In particular, we predict one set of offsets shared by the top-left corners of all categories, and another set shared by the bottom-right corners. For training, we apply the smooth L1 Loss (Girshick, 2015) at ground-truth corner locations:

$$L_{off} = \frac{1}{N} \sum_{k=1}^{N} \text{SmoothL1Loss}\left(\boldsymbol{o}_k, \hat{\boldsymbol{o}}_k\right) \quad (3)$$

3.3 Grouping Corners

Multiple objects may appear in an image, and thus multiple top-left and bottom-right corners may be detected. We need to determine if a pair of the top-left corner and bottom-right corner is from the same bounding box. Our approach is inspired by the Associative Embedding method proposed by Newell et al. (2017) for the task of multi-person pose estimation. Newell et al. detect all human joints and generate an embedding for each detected joint. They group the joints based on the distances between the embeddings.

The idea of associative embedding is also applicable to our task. The network predicts an embedding vector for each detected corner such that if a top-left corner and a bottom-right corner belong to the same bounding box, the distance between their embeddings should

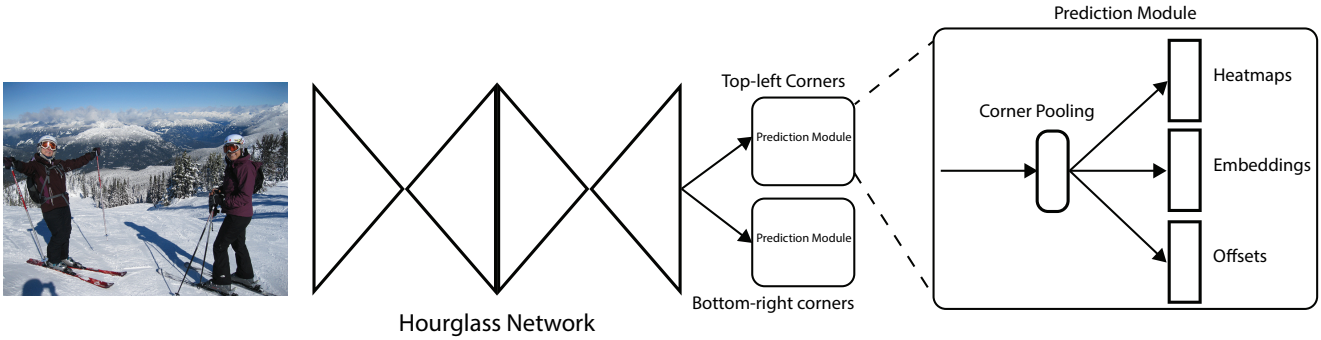


Fig. 4 CornerNet概述。骨干网络之后是两个预测模块，一个用于左上角，另一个用于右下角。th 另一个用于右下角。利用两个模块的预测结果，我们定位并分组这些角点。

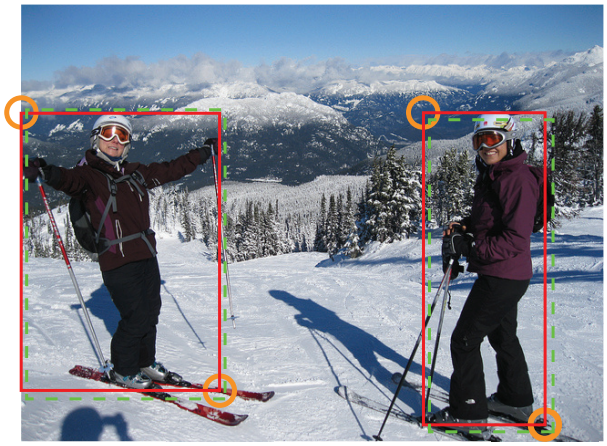


Fig. 5 用于训练的“真实”热图。其角点位于正位置*(orange circles)*半径内的框*(greendotted rectangles)*，仍与真实标注*(red solid rectangles)*存在较大重叠。

$e^{-\frac{x^2}{2\sigma^2}+\frac{y^2}{2\sigma^2}}$，其中心位于正位置，且其$\sigma$为半径的1/3。

令p_{cij}为预测热图中类别c在位置(i,j)的得分，y_{cij}为用未归一化高斯分布增强的“真实”热图。我们设计了一种焦点损失（Lin等人，2017）的变体：

$$L_{det} = \frac{-1}{N}\sum_{c=1}^{C}\sum_{i=1}^{H}\sum_{j=1}^{W}\begin{cases}(1-p_{cij})^{\alpha}\log\left(p_{cij}\right) & \text{若 } y_{cij}=1\\ (1-y_{cij})^{\beta}\left(p_{cij}\right)^{\alpha}\log\left(1-p_{cij}\right) & \text{否则}\end{cases} \quad (1)$$

其中N是图像中的物体数量，α和β是控制各点贡献度的超参数（所有实验中我们将α设为2，β设为4）。通过y_{cij}中编码的高斯凸起，$(1-y_{cij})$项可降低真实位置附近的惩罚强度。

许多网络（He等人，2016；Newell等人，2016）采用下采样层来收集全局信息并减少内存占用。当这些层以全卷积方式应用于图像时，其输出尺寸会减小，但{v*}表示的关键特征得以保留。

输出通常比图像小。因此，图像中的位置(x,y)被映射到热图中的位置$\left(\lfloor\frac{x}{n}\rfloor,\lfloor\frac{y}{n}\rfloor\right)$，其中$n$是下采样因子。当我们将位置从热图重新映射到输入图像时，可能会损失一些精度，这会极大地影响小边界框与其真实标注之间的IoU。为了解决这个问题，我们预测位置偏移，以便在将角点位置重新映射到输入分辨率之前对其进行微调。

$$\boldsymbol{o}_k = \left(\frac{x_k}{n} - \left\lfloor\frac{x_k}{n}\right\rfloor, \frac{y_k}{n} - \left\lfloor\frac{y_k}{n}\right\rfloor\right) \quad (2)$$

其中$\boldsymbol{o}_k$是偏移量，x_k和y_k是角点k的x和y坐标。特别地，我们预测一组由所有类别的左上角共享的偏移量，以及另一组由右下角共享的偏移量。在训练时，我们在真实角点位置应用平滑L1损失函数（Girshick, 2015）：

$$L_{off} = \frac{1}{N}\sum_{k=1}^{N}\text{SmoothL1Loss}\left(\boldsymbol{o}_k, \hat{\boldsymbol{o}}_k\right) \quad (3)$$

3.3 角点分组

一张图像中可能出现多个物体，因此可能检测到多个左上角和右下角。我们需要判断一个左上角和一个右下角是否来自同一个边界框。我们的方法受到Newell等人（2017）为多人姿态估计任务提出的关联嵌入方法的启发。Newell等人检测所有人体关节点并为每个检测到的关节点生成一个嵌入表示。他们根据嵌入表示之间的距离对关节点进行分组。

关联嵌入的思想同样适用于我们的任务。网络为每个检测到的角点预测一个嵌入向量，使得如果左上角点和右下角点属于同一个边界框，它们嵌入之间的距离应当

be small. We can then group the corners based on the distances between the embeddings of the top-left and bottom-right corners. The actual values of the embeddings are unimportant. Only the distances between the embeddings are used to group the corners.

We follow Newell et al. (2017) and use embeddings of 1 dimension. Let e_{t_k} be the embedding for the top-left corner of object k and e_{b_k} for the bottom-right corner. As in Newell and Deng (2017), we use the "pull" loss to train the network to group the corners and the "push" loss to separate the corners:

$$L_{pull} = \frac{1}{N}\sum_{k=1}^{N}\left[\left(e_{t_k} - e_k\right)^2 + \left(e_{b_k} - e_k\right)^2\right], \tag{4}$$

$$L_{push} = \frac{1}{N(N-1)}\sum_{k=1}^{N}\sum_{\substack{j=1\\ j\neq k}}^{N}\max\left(0, \Delta - |e_k - e_j|\right), \tag{5}$$

where e_k is the average of e_{t_k} and e_{b_k} and we set Δ to be 1 in all our experiments. Similar to the offset loss, we only apply the losses at the ground-truth corner location.

3.4 Corner Pooling

As shown in Fig. 2, there is often no local visual evidence for the presence of corners. To determine if a pixel is a top-left corner, we need to look horizontally towards the right for the topmost boundary of an object and vertically towards the bottom for the leftmost boundary. We thus propose *corner pooling* to better localize the corners by encoding explicit prior knowledge.

Suppose we want to determine if a pixel at location (i, j) is a top-left corner. Let f_t and f_l be the feature maps that are the inputs to the top-left corner pooling layer, and let $f_{t_{ij}}$ and $f_{l_{ij}}$ be the vectors at location (i, j) in f_t and f_l respectively. With $H \times W$ feature maps, the corner pooling layer first max-pools all feature vectors between (i, j) and (i, H) in f_t to a feature vector t_{ij}, and max-pools all feature vectors between (i, j) and (W, j) in f_l to a feature vector l_{ij}. Finally, it adds t_{ij} and l_{ij} together. This computation can be expressed by the following equations:

$$t_{ij} = \begin{cases} \max\left(f_{t_{ij}}, t_{(i+1)j}\right) & \text{if } i < H \\ f_{t_{Hj}} & \text{otherwise} \end{cases} \tag{6}$$

$$l_{ij} = \begin{cases} \max\left(f_{l_{ij}}, l_{i(j+1)}\right) & \text{if } j < W \\ f_{l_{iW}} & \text{otherwise} \end{cases} \tag{7}$$

where we apply an elementwise max operation. Both t_{ij} and l_{ij} can be computed efficiently by dynamic programming as shown Fig. 8.

We define bottom-right corner pooling layer in a similar way. It max-pools all feature vectors between $(0, j)$ and (i, j), and all feature vectors between $(i, 0)$ and (i, j) before adding the pooled results. The corner pooling layers are used in the prediction modules to predict heatmaps, embeddings and offsets.

The architecture of the prediction module is shown in Fig. 7. The first part of the module is a modified version of the residual block (He et al., 2016). In this modified residual block, we replace the first 3×3 convolution module with a corner pooling module, which first processes the features from the backbone network by two 3×3 convolution modules [1] with 128 channels and then applies a corner pooling layer. Following the design of a residual block, we then feed the pooled features into a 3×3 Conv-BN layer with 256 channels and add back the projection shortcut. The modified residual block is followed by a 3×3 convolution module with 256 channels, and 3 Conv-ReLU-Conv layers to produce the heatmaps, embeddings and offsets.

3.5 Hourglass Network

CornerNet uses the hourglass network (Newell et al., 2016) as its backbone network. The hourglass network was first introduced for the human pose estimation task. It is a fully convolutional neural network that consists of one or more hourglass modules. An hourglass module first downsamples the input features by a series of convolution and max pooling layers. It then upsamples the features back to the original resolution by a series of upsampling and convolution layers. Since details are lost in the max pooling layers, skip layers are added to bring back the details to the upsampled features. The hourglass module captures both global and local features in a single unified structure. When multiple hourglass modules are stacked in the network, the hourglass modules can reprocess the features to capture higher-level of information. These properties make the hourglass network an ideal choice for object detection as well. In fact, many current detectors (Shrivastava et al., 2016; Fu et al., 2017; Lin et al., 2016; Kong et al., 2017) already adopted networks similar to the hourglass network.

Our hourglass network consists of two hourglasses, and we make some modifications to the architecture of the hourglass module. Instead of using max pool-

[1] Unless otherwise specified, our convolution module consists of a convolution layer, a BN layer (Ioffe and Szegedy, 2015) and a ReLU layer

很小。然后，我们可以根据左上角和右下角嵌入之间的距离对角落进行分组。嵌入的实际值并不重要，仅使用嵌入之间的距离来对角落进行分组。

我们遵循Newell等人（2017）的方法，使用1维嵌入。令e_{t_k}表示物体k左上角的嵌入，e_{b_k}表示右下角的嵌入。如Newell与Deng（2017）所述，我们使用“拉近”损失训练网络以聚合角点，并使用“推远”损失以分离角点：

$$L_{pull} = \frac{1}{N}\sum_{k=1}^{N}\left[\left(e_{t_k} - e_k\right)^2 + \left(e_{b_k} - e_k\right)^2\right], \tag{4}$$

$$L_{push} = \frac{1}{N(N-1)}\sum_{k=1}^{N}\sum_{\substack{j=1\\ j\neq k}}^{N}\max\left(0, \Delta - |e_k - e_j|\right), \tag{5}$$

其中e_k是e_{t_k}和e_{b_k}的平均值，我们在所有实验中均将Δ设为1。与偏移损失类似，我们仅在真实角点位置应用这些损失。

3.4 角点池化

如图2所示，角落的存在通常缺乏局部视觉证据。要判断一个像素是否为左上角，我们需要水平向右寻找物体的最上边界，并垂直向下寻找最左边界。因此，我们提出{v*}，通过编码显式先验知识来更精准地定位角落。

假设我们要判断位置 (i, j) 的像素是否为左上角点。令 f_t 和 f_l 作为输入至左上角池化层的特征图，并设 $f_{t_{ij}}$ 和 $f_{l_{ij}}$ 分别为 f_t 和 f_l 中位置 (i, j) 的向量。在 $H \times W$ 个特征图的情况下，角点池化层首先将 f_t 中 (i, j) 到 (i, H) 之间的所有特征向量最大池化为特征向量 t_{ij}，并将 f_l 中 (i, j) 到 (W, j) 之间的所有特征向量最大池化为特征向量 l_{ij}。最后，将 t_{ij} 和 l_{ij} 相加。该计算过程可通过以下公式表示：

$$t_{ij} = \begin{cases} \max\left(f_{t_{ij}}, t_{(i+1)j}\right) & \text{if } i < H \\ f_{t_{Hj}} & \text{otherwise} \end{cases} \tag{6}$$

$$l_{ij} = \begin{cases} \max\left(f_{l_{ij}}, l_{i(j+1)}\right) & \text{if } j < W \\ f_{l_{iW}} & \text{otherwise} \end{cases} \tag{7}$$

其中我们应用了逐元素的最大化操作。如图8所示，t_{ij}和l_{ij}都可以通过动态规划高效计算。

我们以类似的方式定义右下角池化层。它在将池化结果相加之前，对$(0, j)$到(i, j)之间的所有特征向量以及$(i,0)$到(i, j)之间的所有特征向量进行最大池化。角池化层被用于预测模块中，以预测热图、嵌入向量和偏移量。

预测模块的架构如图7所示。该模块的第一部分是残差块（He等人，2016）的改进版本。在这个改进的残差块中，我们将第一个3×3卷积模块替换为角点池化模块，该模块首先通过两个128通道的3×3卷积模块处理来自骨干网络的特征，然后应用角点池化层。遵循残差块的设计，随后将池化后的特征输入一个256通道的3×3 Conv-BN层，并添加投影捷径。改进的残差块后接一个256通道的3×3卷积模块，以及3个Conv-ReLU-Conv层，用于生成热图、嵌入向量和偏移量。

3.5 沙漏网络

CornerNet采用沙漏网络（Newell等人，2016）作为其骨干网络。沙漏网络最初是为人体姿态估计任务提出的。它是一个全卷积神经网络，包含一个或多个沙漏模块。沙漏模块首先通过一系列卷积和最大池化层对输入特征进行下采样，随后通过一系列上采样和卷积层将特征上采样回原始分辨率。由于最大池化层会丢失细节信息，网络通过添加跳跃连接层将细节信息重新注入上采样特征。沙漏模块在单一统一结构中同时捕获全局和局部特征。当多个沙漏模块堆叠在网络中时，这些模块能对特征进行再处理以捕捉更高层次的信息。这些特性使沙漏网络也成为目标检测的理想选择。事实上，许多当前检测器（Shrivastava等人，2016；Fu等人，2017；Lin等人，2016；Kong等人，2017）已采用类似沙漏网络的架构。

我们的沙漏网络由两个沙漏组成，并对沙漏模块的架构进行了一些修改。我们不再使用最大池化——

[1] Unless otherwise specified, our convolution module consists of a convolution layer, a BN layer (Ioffe and Szegedy, 2015) and a ReLU layer

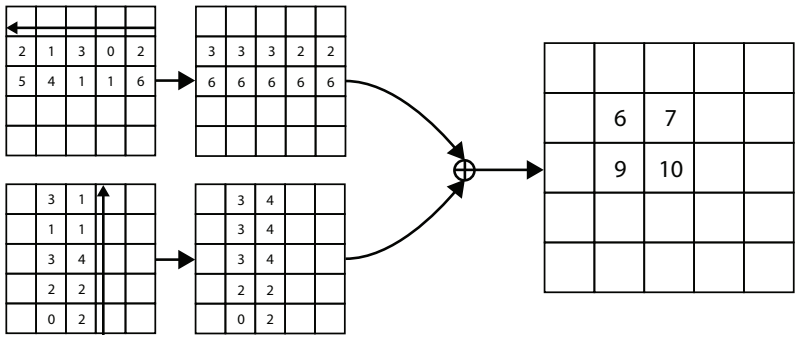


Fig. 6 The top-left corner pooling layer can be implemented very efficiently. We scan from right to left for the horizontal max-pooling and from bottom to top for the vertical max-pooling. We then add two max-pooled feature maps.

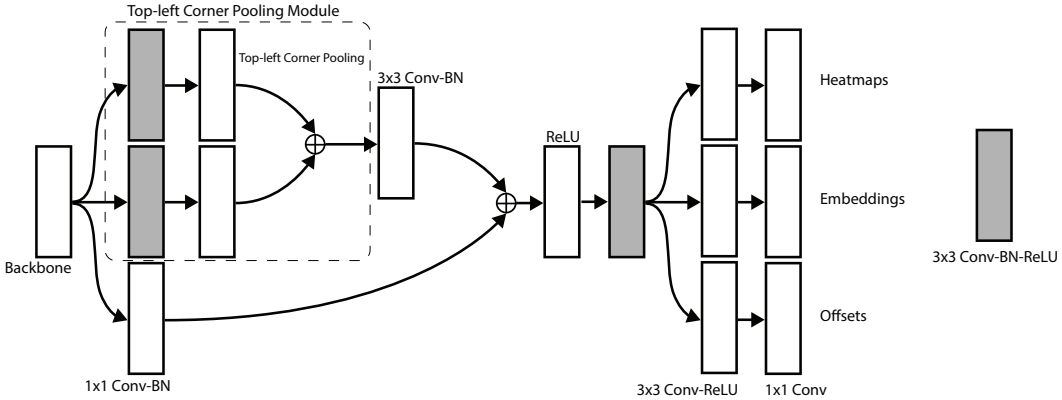


Fig. 7 The prediction module starts with a modified residual block, in which we replace the first convolution module with our corner pooling module. The modified residual block is then followed by a convolution module. We have multiple branches for predicting the heatmaps, embeddings and offsets.

ing, we simply use stride 2 to reduce feature resolution. We reduce feature resolutions 5 times and increase the number of feature channels along the way $(256, 384, 384, 384, 512)$. When we upsample the features, we apply 2 residual modules followed by a nearest neighbor upsampling. Every skip connection also consists of 2 residual modules. There are 4 residual modules with 512 channels in the middle of an hourglass module. Before the hourglass modules, we reduce the image resolution by 4 times using a 7×7 convolution module with stride 2 and 128 channels followed by a residual block (He et al., 2016) with stride 2 and 256 channels.

Following (Newell et al., 2016), we also add intermediate supervision in training. However, we do not add back the intermediate predictions to the network as we find that this hurts the performance of the network. We apply a 1×1 Conv-BN module to both the input and output of the first hourglass module. We then merge them by element-wise addition followed by a ReLU and a residual block with 256 channels, which is then used as the input to the second hourglass module. The depth of the hourglass network is 104. Unlike many other state-of-the-art detectors, we only use the features from the last layer of the whole network to make predictions.

4 Experiments

4.1 Training Details

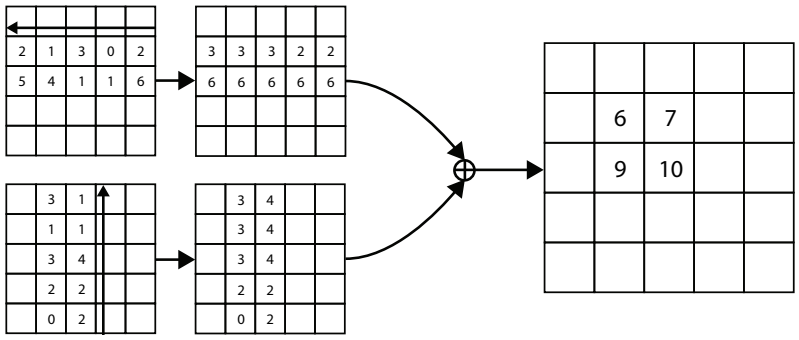
We implement CornerNet in PyTorch (Paszke et al., 2017). The network is randomly initialized under the default setting of PyTorch with no pretraining on any external dataset. As we apply focal loss, we follow (Lin et al., 2017) to set the biases in the convolution layers that predict the corner heatmaps. During training, we set the input resolution of the network to 511×511, which leads to an output resolution of 128×128. To reduce overfitting, we adopt standard data augmentation techniques including random horizontal flipping, random scaling, random cropping and random color jittering, which includes adjusting the brightness, saturation and contrast of an image. Finally, we apply PCA (Krizhevsky et al., 2012) to the input image.

We use Adam (Kingma and Ba, 2014) to optimize the full training loss:

$$L = L_{det} + \alpha L_{pull} + \beta L_{push} + \gamma L_{off} \tag{8}$$

where α, β and γ are the weights for the pull, push and offset loss respectively. We set both α and β to 0.1 and

F



ig. 6 左上角池化层可以实现得非常高效。我们通过从右向左扫描来实现水平方向的米ax-pooling和从下到上的垂直max-pooling。然后我们将两个最大池化后的特征图相加。

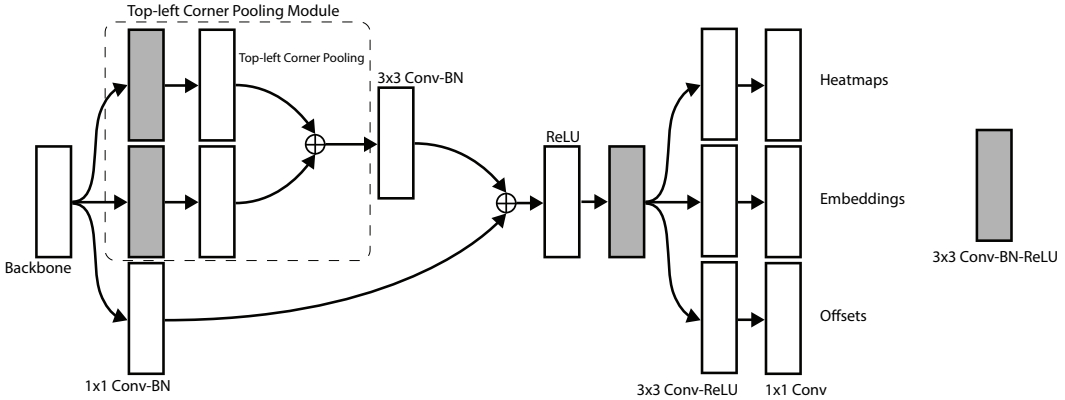


Fig. 7 预测模块以一个改进的残差块开始，其中我们将第一个卷积模块替换为我们的角点池化模块。改进的残差块之后是一个卷积模块。我们拥有多个分支，用于预测热图、嵌入向量和偏移量。

在编码过程中，我们直接采用步长为2的操作来降低特征分辨率。我们共进行了5次特征分辨率缩减，并在此过程中逐步增加特征通道数（256, → 384, → 384, → 384 , → 512）。当对特征进行上采样时，我们会先应用2个残差模块，再进行最近邻上采样。每个跳跃连接同样包含2个残差模块。在沙漏模块的中部，共有4个通道数为512的残差模块。在进入沙漏模块之前，我们通过一个步长为2、通道数为128的7×7卷积模块将图像分辨率降低4倍，随后接一个步长为2、通道数为256的残差块（He et al., 2016）。

遵循（Newell等人，2016）的做法，我们在训练中也加入了中间监督。然而，我们并未将中间预测结果重新加入网络，因为我们发现这会损害网络的性能。我们对第一个沙漏模块的输入和输出均应用一个1×1的Conv-BN模块。随后，我们通过逐元素相加的方式合并它们，接着经过一个ReLU激活函数和一个具有256个通道的残差块处理，其结果作为第二个沙漏模块的输入。该沙漏网络的深度为104层。与许多其他先进检测器不同，我们仅使用整个网络最后一层的特征进行预测。

4 Experiments

4.1 训练细节

我们在PyTorch（Paszke等人，2017）中实现了CornerNet。网络在PyTorch的默认设置下随机初始化，未在任何外部数据集上进行预训练。由于我们应用了焦点损失，我们遵循（Lin等人，2017）的方法来设置预测角点热图的卷积层中的偏置。在训练期间，我们将网络的输入分辨率设置为511×511，这导致输出分辨率为128×128。为了减少过拟合，我们采用了标准的数据增强技术，包括随机水平翻转、随机缩放、随机裁剪和随机颜色抖动（包括调整图像的亮度、饱和度和对比度）。最后，我们对输入图像应用了PCA（Krizhevsky等人，2012）。

我们使用Adam（Kingma和Ba，2014）来优化完整的训练损失：

$$L = L_{det} + \alpha L_{pull} + \beta L_{push} + \gamma L_{off} \tag{8}$$

其中 α、β 和 γ 分别为拉力、推力和偏移损失的权重。我们将 α 和 β 均设为 0.1，且

Table 1 Ablation on corner pooling on MS COCO validation.

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
w/o corner pooling	36.5	52.0	38.8	17.5	38.9	49.4
w/ corner pooling	38.4	53.8	40.9	18.6	40.5	51.8
improvement	+2.0	+2.1	+2.1	+1.1	+2.4	+3.6

Table 2 Reducing the penalty given to the negative locations near positive locations helps significantly improve the performance of the network

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
w/o reducing penalty	32.9	49.1	34.8	19.0	37.0	40.7
fixed radius	35.6	52.5	37.7	18.7	38.5	46.0
object-dependent radius	38.4	53.8	40.9	18.6	40.5	51.8

Table 3 Corner pooling consistently improves the network performance on detecting corners in different image quadrants, showing that corner pooling is effective and stable over both small and large areas.

	mAP w/o pooling	mAP w/ pooling	improvement
Top-Left Corners			
Top-Left Quad.	66.1	69.2	+3.1
Bottom-Right Quad.	60.8	63.5	+2.7
Bottom-Right Corners			
Top-Left Quad.	53.4	56.2	+2.8
Bottom-Right Quad.	65.0	67.6	+2.6

γ to 1. We find that 1 or larger values of α and β lead to poor performance. We use a batch size of 49 and train the network on 10 Titan X (PASCAL) GPUs (4 images on the master GPU, 5 images per GPU for the rest of the GPUs). To conserve GPU resources, in our ablation experiments, we train the networks for 250k iterations with a learning rate of 2.5×10^{-4}. When we compare our results with other detectors, we train the networks for an extra 250k iterations and reduce the learning rate to 2.5×10^{-5} for the last 50k iterations.

4.2 Testing Details

During testing, we use a simple post-processing algorithm to generate bounding boxes from the heatmaps, embeddings and offsets. We first apply non-maximal suppression (NMS) by using a 3×3 max pooling layer on the corner heatmaps. Then we pick the top 100 top-left and top 100 bottom-right corners from the heatmaps. The corner locations are adjusted by the corresponding offsets. We calculate the L1 distances between the embeddings of the top-left and bottom-right corners. Pairs that have distances greater than 0.5 or contain corners from different categories are rejected. The average scores of the top-left and bottom-right corners are used as the detection scores.

Instead of resizing an image to a fixed size, we maintain the original resolution of the image and pad it with zeros before feeding it to CornerNet. Both the original and flipped images are used for testing. We combine the detections from the original and flipped images, and apply soft-nms (Bodla et al., 2017) to suppress redundant detections. Only the top 100 detections are reported. The average inference time is 244ms per image on a Titan X (PASCAL) GPU.

4.3 MS COCO

We evaluate CornerNet on the very challenging MS COCO dataset (Lin et al., 2014). MS COCO contains 80k images for training, 40k for validation and 20k for testing. All images in the training set and 35k images in the validation set are used for training. The remaining 5k images in validation set are used for hyper-parameter searching and ablation study. All results on the test set are submitted to an external server for evaluation. To provide fair comparisons with other detectors, we report our main results on the test-dev set. MS COCO uses average precisions (APs) at different IoUs and APs for different object sizes as the main evaluation metrics.

Table 1 在MS COCO验证集上对角点池化的消融实验 n。

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
w/o corner pooling	36.5	52.0	38.8	17.5	38.9	49.4
w/ corner pooling	38.4	53.8	40.9	18.6	40.5	51.8
improvement	+2.0	+2.1	+2.1	+1.1	+2.4	+3.6

Table 2 减少对正位置附近负位置的惩罚有助于显著提升网络性能。

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
w/o reducing penalty	32.9	49.1	34.8	19.0	37.0	40.7
fixed radius	35.6	52.5	37.7	18.7	38.5	46.0
object-dependent radius	38.4	53.8	40.9	18.6	40.5	51.8

Table 3 角点池化在不同图像象限中检测角点时持续提升网络性能，这表明角点池化无论在小区域还是大区域都具有稳定且高效的效果。

	mAP w/o pooling	mAP w/ pooling	improvement
Top-Left Corners			
Top-Left Quad.	66.1	69.2	+3.1
Bottom-Right Quad.	60.8	63.5	+2.7
Bottom-Right Corners			
Top-Left Quad.	53.4	56.2	+2.8
Bottom-Right Quad.	65.0	67.6	+2.6

γ 我们发现，α和β的值设为1或更大时会导致性能不佳。我们使用49的批次大小，并在10块Titan X（PASCAL）GPU上训练网络（主GPU处理4张图像，其余GPU各处理5张图像）。为节省GPU资源，在消融实验中，我们以2.5×10^{-4}的学习率训练网络25万次迭代。当与其他检测器比较结果时，我们会额外训练25万次迭代，并在最后5万次迭代中将学习率降至2.5×10^{-5}。

在将图像输入CornerNet之前，我们先用零填充图像。测试时同时使用原始图像和翻转后的图像。我们将原始图像和翻转图像的检测结果合并，并应用软性非极大值抑制（Bodla等人，2017）来抑制冗余检测。仅报告前100个检测结果。在Titan X（PASCAL）GPU上，平均每张图像的推理时间为244毫秒。

4.2 测试详情

在测试过程中，我们采用一种简单的后处理算法，根据热图、嵌入向量和偏移量生成边界框。我们首先在角点热图上应用3×3最大池化层进行非极大值抑制（NMS）。随后从热图中选取前100个左上角点和前100个右下角点，并根据对应的偏移量调整角点位置。通过计算左上角与右下角嵌入向量的L1距离，我们筛除距离大于0.5或包含不同类别角点的配对。最终以左上角与右下角得分的平均值作为检测置信度。

我们不再将图像调整为固定尺寸，而是保持图像原有的分辨率，并通过填充来

4.3 MS COCO

我们在极具挑战性的MS COCO数据集（Lin等人，2014）上评估CornerNet。MS COCO包含8万张训练图像、4万张验证图像和2万张测试图像。训练集中的所有图像以及验证集中的3.5万张图像被用于训练。验证集中剩余的5千张图像用于超参数搜索和消融实验。测试集上的所有结果均提交至外部服务器进行评估。为与其他检测器进行公平比较，我们在test-dev集上报告主要结果。MS COCO采用不同IoU阈值下的平均精度（AP）以及针对不同目标尺寸的AP作为主要评估指标。

Fig. 8 Qualitative examples showing corner pooling helps better localize the corners.

Table 4 The hourglass network is crucial to the performance of CornerNet.

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
FPN (w/ ResNet-101) + Corners	30.2	44.1	32.0	13.3	33.3	42.7
Hourglass + Anchors	32.9	53.1	35.6	16.5	38.5	45.0
Hourglass + Corners	38.4	53.8	40.9	18.6	40.5	51.8

4.4 Ablation Study

4.4.1 Corner Pooling

Corner pooling is a key component of CornerNet. To understand its contribution to performance, we train another network without corner pooling but with the same number of parameters.

Tab. 1 shows that adding corner pooling gives significant improvement: 2.0% on AP, 2.1% on AP^{50} and 2.1% on AP^{75}. We also see that corner pooling is especially helpful for medium and large objects, improving their APs by 2.4% and 3.6% respectively. This is expected because the topmost, bottommost, leftmost, rightmost boundaries of medium and large objects are likely to be further away from the corner locations. Fig. 8 shows four qualitative examples with and without corner pooling.

4.4.2 Stability of Corner Pooling over Larger Area

Corner pooling pools over different sizes of area in different quadrants of an image. For example, the top-left corner pooling pools over larger areas both horizontally and vertically in the upper-left quadrant of an image, compared to the lower-right quadrant. Therefore, the location of a corner may affect the stability of the corner pooling.

We evaluate the performance of our network on detecting both the top-left and bottom-right corners in different quadrants of an image. Detecting corners can be seen as a binary classification task i.e. the ground-truth location of a corner is positive, and any location outside of a small radius of the corner is negative. We measure the performance using mAPs over all categories on the MS COCO validation set.

Tab. 3 shows that without corner pooling, the top-left corner mAPs of upper-left and lower-right quadrant are 66.1% and 60.8% respectively. Top-left corner pooling improves the mAPs by 3.1% (to 69.2%) and 2.7% (to 63.5%) respectively. Similarly, bottom-right corner pooling improves the bottom-right corner mAPs of upper-left quadrant by 2.8% (from 53.4% to 56.2%), and lower-right quadrant by 2.6% (from 65.0% to 67.6%). Corner pooling gives similar improvement to corners at different quadrants, show that corner pooling is effective and stable over both small and large areas.

4.4.3 Reducing Penalty to Negative Locations

We reduce the penalty given to negative locations around a positive location, within a radius determined by the size of the object (Sec. 3.2). To understand how this helps train CornerNet, we train one network with no penalty reduction and another network with a fixed radius of 2.5. We compare them with CornerNet on the validation set.

Tab. 2 shows that a fixed radius improves AP over the baseline by 2.7%, AP^m by 1.5% and AP^l by 5.3%. Object-dependent radius further improves the AP by

Fig. 8 展示角点池化有助于更好地定位角点的定性示例。

Table 4 T沙漏网络对于CornerNet的性能至关重要 等。

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
FPN (w/ ResNet-101) + Corners	30.2	44.1	32.0	13.3	33.3	42.7
Hourglass + Anchors	32.9	53.1	35.6	16.5	38.5	45.0
Hourglass + Corners	38.4	53.8	40.9	18.6	40.5	51.8

4.4 消融研究

4.4.1 Corner Pooling

角点池化是CornerNet的关键组成部分。为了理解它对性能的贡献，我们训练了另一个没有角点池化但参数数量相同的网络。

表1显示，添加角点池化带来了显著提升：AP提升2.0%，AP^{50}提升2.1%，AP^{75}提升2.1%。我们还发现角点池化对中型和大型物体尤其有效，分别将其AP提升了2.4%和3.6%。这符合预期，因为中型和大型物体的最顶部、最底部、最左侧和最右侧边界往往距离角点位置更远。图8展示了使用与未使用角点池化的四个定性示例。

4.4.2 Stability of Corner Pooling over Larger Area

角点池化在图像的不同象限内对不同大小的区域进行池化。例如，与右下象限相比，左上角池化在图像的左上象限中在水平和垂直方向上对更大的区域进行池化。因此，角点的位置可能会影响角点池化的稳定性。

我们评估了网络在检测左上角和右下角两方面的性能图像的不同象限。角点检测可视为一个二元分类任务，即角点的真实位置为正样本，而角点小半径范围外的任何位置均为负样本。我们使用MS COCO验证集上所有类别的平均精度均值（mAP）来衡量性能。

表3显示，在不使用角点池化的情况下，左上象限和右下象限的左上角mAP分别为66.1%和60.8%。左上角池化分别将mAP提升了3.1%（至69.2%）和2.7%（至63.5%）。类似地，右下角池化将左上象限的右下角mAP提升了2.8%（从53.4%至56.2%），并将右下象限的右下角mAP提升了2.6%（从65.0%至67.6%）。角点池化对不同象限的角点均带来相似的改进，表明角点池化在不同大小区域上均具有有效且稳定的性能。

4.4.3 Reducing Penalty to Negative Locations

我们减少了对正位置周围负位置的惩罚，惩罚半径由物体大小决定（第3.2节）。为了理解这如何帮助训练CornerNet，我们训练了一个没有惩罚减少的网络和另一个固定半径为2.5的网络。我们在验证集上将它们与CornerNet进行了比较。

表2显示，固定半径将AP较基线提升了2.7%，AP^m提升了1.5%，AP^l提升了5.3%。而物体相关半径进一步将AP提升了

Table 5 CornerNet performs much better at high IoUs than other state-of-the-art detectors.

	AP	AP^{50}	AP^{60}	AP^{70}	AP^{80}	AP^{90}
RetinaNet (Lin et al., 2017)	39.8	59.5	55.6	48.2	36.4	15.1
Cascade R-CNN (Cai and Vasconcelos, 2017)	38.9	57.8	53.4	46.9	35.8	15.8
Cascade R-CNN + IoU Net (Jiang et al., 2018)	41.4	59.3	55.3	49.6	39.4	19.5
CornerNet	40.6	56.1	52.0	46.8	38.8	23.4

Table 6 Error analysis. We replace the predicted heatmaps and offsets with the ground-truth values. Using the ground-truth heatmaps alone improves the AP from 38.4% to 73.1%, suggesting that the main bottleneck of CornerNet is detecting corners.

	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l
	38.4	53.8	40.9	18.6	40.5	51.8
w/ gt heatmaps	73.1	87.7	78.4	60.9	81.2	81.8
w/ gt heatmaps + offsets	86.1	88.9	85.5	84.8	87.2	82.0

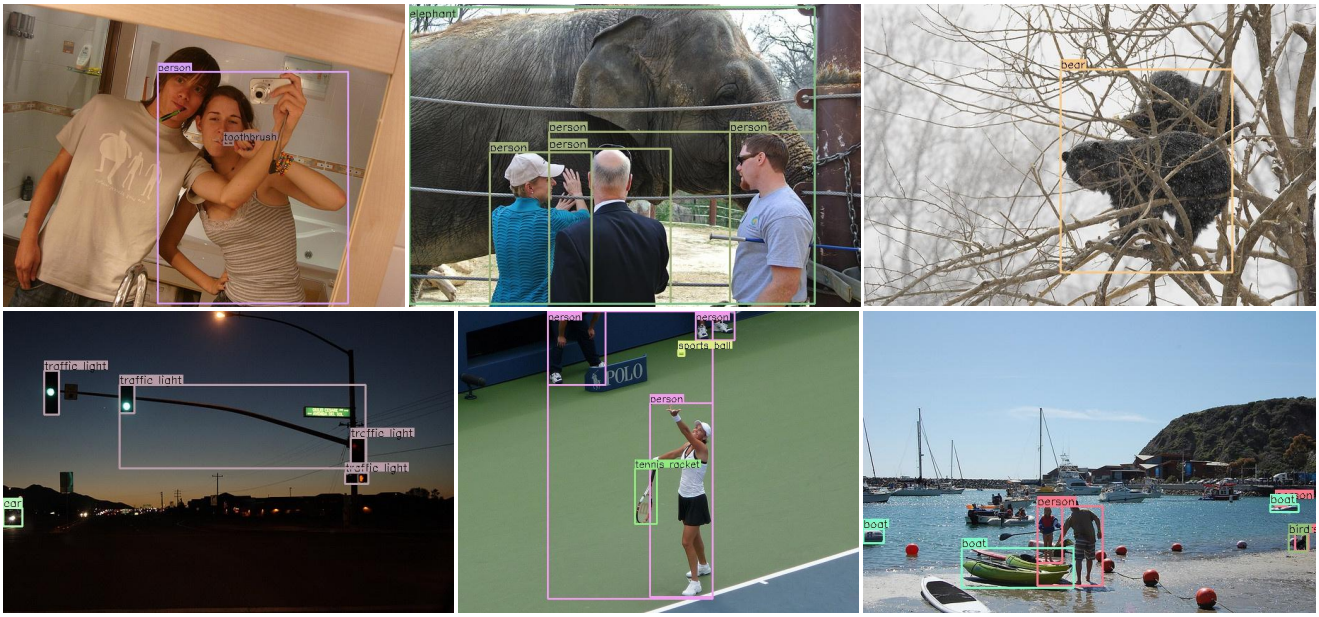


Fig. 9 Qualitative example showing errors in predicting corners and embeddings. The first row shows images where CornerNet mistakenly combines boundary evidence from different objects. The second row shows images where CornerNet predicts similar embeddings for corners from different objects.

2.8%, AP^m by 2.0% and AP^l by 5.8%. In addition, we see that the penalty reduction especially benefits medium and large objects.

4.4.4 Hourglass Network

CornerNet uses the hourglass network (Newell et al., 2016) as its backbone network. Since the hourglass network is not commonly used in other state-of-the-art detectors, we perform an experiment to study the contribution of the hourglass network in CornerNet. We train a CornerNet in which we replace the hourglass network with FPN (w/ ResNet-101) (Lin et al., 2017), which is more commonly used in state-of-the-art object detectors. We only use the final output of FPN for predictions. Meanwhile, we train an anchor box based detector which uses the hourglass network as its backbone. Each hourglass module predicts anchor boxes at multiple resolutions by using features at multiple scales during upsampling stage. We follow the anchor box design in RetinaNet (Lin et al., 2017) and add intermediate supervisions during training. In both experiments, we initialize the networks from scratch and follow the same training procedure as we train CornerNet (Sec. 4.1).

Tab. 4 shows that CornerNet with hourglass network outperforms CornerNet with FPN by 8.2% AP, and the anchor box based detector with hourglass network by 5.5% AP. The results suggest that the choice of the backbone network is important and the hourglass network is crucial to the performance of CornerNet.

Table 5 角邻　　在高IoU条件下，t的表现远优于其他最先进的方法。　　rt探测器。

	AP	AP^{50}	AP^{60}	AP^{70}	AP^{80}	AP^{90}
RetinaNet (Lin et al., 2017)	39.8	59.5	55.6	48.2	36.4	15.1
Cascade R-CNN (Cai and Vasconcelos, 2017)	38.9	57.8	53.4	46.9	35.8	15.8
Cascade R-CNN + IoU Net (Jiang et al., 2018)	41.4	59.3	55.3	49.6	39.4	19.5
CornerNet	40.6	56.1	52.0	46.8	38.8	23.4

Table 6 误差分析。我们将预测的热图和偏移量替换为真实值。使用真实值
h仅使用热图就将AP从38.4%提升至73.1%，这表明CornerNet的主要瓶颈在于检测角点。

	AP	AP^{50}	AP^{75}	AP^{s}	AP^{m}	AP^{l}
	38.4	53.8	40.9	18.6	40.5	51.8
w/ gt heatmaps	73.1	87.7	78.4	60.9	81.2	81.8
w/ gt heatmaps + offsets	86.1	88.9	85.5	84.8	87.2	82.0

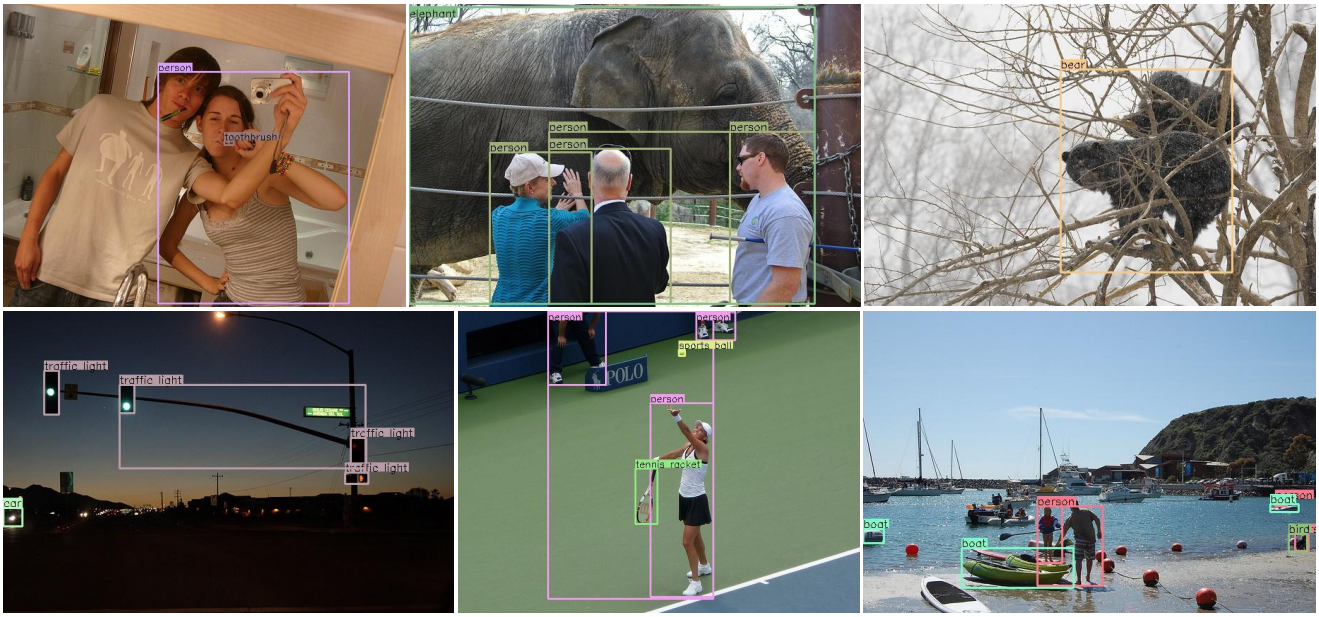


Fig. 9 定性示例展示了预测角点和嵌入时的错误。第一行图像中，CornerNet错误地融合了来自不同物体的边界证据。第二行图像中，CornerNet为不同物体的角点预测了相似的嵌入。

2.8%，AP^{m}提升2.0%，AP^{l}提升5.8%。此外，我们发现惩罚机制的减少尤其对中型和大型物体检测有益。

4.4.4 Hourglass Network

CornerNet采用沙漏网络（Newell等人，2016）作为其骨干网络。由于沙漏网络在其他先进检测器中并不常用，我们通过实验来研究沙漏网络在CornerNet中的贡献。我们训练了一个将沙漏网络替换为FPN（基于Res Net-101）（Lin等人，2017）的CornerNet版本，后者在先进目标检测器中更为常用。我们仅使用FPN的最终输出进行预测。同时，我们训练了一个基于锚框的检测器——该网络采用沙漏网络作为其骨干。每个沙漏模块在上采样阶段利用多尺度特征，以预测多分辨率的锚框。我们遵循RetinaNet（Lin等人，2017）中的锚框设计，并在训练过程中添加了中间监督。在两项实验中，我们均从头开始初始化网络，并遵循与训练CornerNet（第4.1节）相同的训练流程。

表4显示，使用沙漏网络的CornerNet比使用FPN的CornerNet在AP上高出8.2%，比基于锚框且使用沙漏网络的检测器高出5.5%。结果表明，骨干网络的选择至关重要，而沙漏网络对CornerNet的性能起着关键作用。

Table 7 CornerNet versus others on MS COCO test-dev. CornerNet outperforms all one-stage detectors and achieves results competitive to two-stage detectors

Method	Backbone	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l	AR^1	AR^{10}	AR^{100}	AR^s	AR^m	AR^l
Two-stage detectors													
DeNet (Tychsen-Smith and Petersson, 2017a)	ResNet-101	33.8	53.4	36.1	12.3	36.1	50.8	29.6	42.6	43.5	19.2	46.9	64.3
CoupleNet (Zhu et al., 2017)	ResNet-101	34.4	54.8	37.2	13.4	38.1	50.8	30.0	45.0	46.4	20.7	53.1	68.5
Faster R-CNN by G-RMI (Huang et al., 2017)	Inception-ResNet-v2 (Szegedy et al., 2017)	34.7	55.5	36.7	13.5	38.1	52.0	-	-	-	-	-	-
Faster R-CNN+++ (He et al., 2016)	ResNet-101	34.9	55.7	37.4	15.6	38.7	50.9	-	-	-	-	-	-
Faster R-CNN w/ FPN (Lin et al., 2016)	ResNet-101	36.2	59.1	39.0	18.2	39.0	48.2	-	-	-	-	-	-
Faster R-CNN w/ TDM (Shrivastava et al., 2016)	Inception-ResNet-v2	36.8	57.7	39.2	16.2	39.8	52.1	31.6	49.3	51.9	28.1	56.6	71.1
D-FCN (Dai et al., 2017)	Aligned-Inception-ResNet	37.5	58.0	-	19.4	40.1	52.5	-	-	-	-	-	-
Regionlets (Xu et al., 2017)	ResNet-101	39.3	59.8	-	21.7	43.7	50.9	-	-	-	-	-	-
Mask R-CNN (He et al., 2017)	ResNeXt-101	39.8	62.3	43.4	22.1	43.2	51.2	-	-	-	-	-	-
Soft-NMS (Bodla et al., 2017)	Aligned-Inception-ResNet	40.9	62.8	-	23.3	43.6	53.3	-	-	-	-	-	-
LH R-CNN (Li et al., 2017)	ResNet-101	41.5	-	-	25.2	45.3	53.1	-	-	-	-	-	-
Fitness-NMS (Tychsen-Smith and Petersson, 2017b)	ResNet-101	41.8	60.9	44.9	21.5	45.0	57.5	-	-	-	-	-	-
Cascade R-CNN (Cai and Vasconcelos, 2017)	ResNet-101	42.8	62.1	46.3	23.7	45.5	55.2	-	-	-	-	-	-
D-RFCN + SNIP (Singh and Davis, 2017)	DPN-98 (Chen et al., 2017)	45.7	67.3	51.1	29.3	48.8	57.1	-	-	-	-	-	-
One-stage detectors													
YOLOv2 (Redmon and Farhadi, 2016)	DarkNet-19	21.6	44.0	19.2	5.0	22.4	35.5	20.7	31.6	33.3	9.8	36.5	54.4
DSOD300 (Shen et al., 2017a)	DS/64-192-48-1	29.3	47.3	30.6	9.4	31.5	47.0	27.3	40.7	43.0	16.7	47.1	65.0
GRP-DSOD320 (Shen et al., 2017b)	DS/64-192-48-1	30.0	47.9	31.8	10.9	33.6	46.3	28.0	42.1	44.5	18.8	49.1	65.0
SSD513 (Liu et al., 2016)	ResNet-101	31.2	50.4	33.3	10.2	34.5	49.8	28.3	42.1	44.4	17.6	49.2	65.8
DSSD513 (Fu et al., 2017)	ResNet-101	33.2	53.3	35.2	13.0	35.4	51.1	28.9	43.5	46.2	21.8	49.1	66.4
RefineDet512 (single scale) (Zhang et al., 2017)	ResNet-101	36.4	57.5	39.5	16.6	39.9	51.4	-	-	-	-	-	-
RetinaNet800 (Lin et al., 2017)	ResNet-101	39.1	59.1	42.3	21.8	42.7	50.2	-	-	-	-	-	-
RefineDet512 (multi scale) (Zhang et al., 2017)	ResNet-101	41.8	62.9	45.7	25.6	45.1	54.1	-	-	-	-	-	-
CornerNet511 (single scale)	Hourglass-104	40.6	56.4	43.2	19.1	42.8	54.3	35.3	54.7	59.4	37.4	62.4	77.2
CornerNet511 (multi scale)	Hourglass-104	42.2	57.8	45.2	20.7	44.8	56.6	36.6	55.9	60.3	39.5	63.2	77.3

Fig. 10 Example bounding box predictions overlaid on predicted heatmaps of corners.

4.4.5 Quality of the Bounding Boxes

A good detector should predict high quality bounding boxes that cover objects tightly. To understand the quality of the bounding boxes predicted by CornerNet, we evaluate the performance of CornerNet at multiple IoU thresholds, and compare the results with other state-of-the-art detectors, including RetinaNet (Lin et al., 2017), Cascade R-CNN (Cai and Vasconcelos, 2017) and IoU-Net (Jiang et al., 2018).

Tab. 5 shows that CornerNet achieves a much higher AP at 0.9 IoU than other detectors, outperforming Cascade R-CNN + IoU-Net by 3.9%, Cascade R-CNN by 7.6% and RetinaNet [2] by 7.3%. This suggests that CornerNet is able to generate bounding boxes of higher quality compared to other state-of-the-art detectors.

4.4.6 Error Analysis

CornerNet simultaneously outputs heatmaps, offsets, and embeddings, all of which affect detection performance. An object will be missed if either corner is missed; precise offsets are needed to generate tight bounding boxes; incorrect embeddings will result in many false bounding boxes. To understand how each part contributes to the final error, we perform an error analysis by replacing the predicted heatmaps and offsets with the ground-truth values and evaluting performance on the validation set.

Tab. 6 shows that using the ground-truth corner heatmaps alone improves the AP from 38.4% to 73.1%.

[2] We use the best model publicly available on `https://github.com/facebookresearch/Detectron/blob/master/MODEL_ZOO.md`

Table 7 CornerNet与其它方法在MS COCO test-dev数据集上的对比。CornerNet超越了所有单阶段检测器并取得了{v*}的优异结果c与两阶段检测器相竞争

Method	Backbone	AP	AP^{50}	AP^{75}	AP^s	AP^m	AP^l	AR^1	AR^{10}	AR^{100}	AR^s	AR^m	AR^l
Two-stage detectors													
DeNet (Tychsen-Smith and Petersson, 2017a)	ResNet-101	33.8	53.4	36.1	12.3	36.1	50.8	29.6	42.6	43.5	19.2	46.9	64.3
CoupleNet (Zhu et al., 2017)	ResNet-101	34.4	54.8	37.2	13.4	38.1	50.8	30.0	45.0	46.4	20.7	53.1	68.5
Faster R-CNN by G-RMI (Huang et al., 2017)	Inception-ResNet-v2 (Szegedy et al., 2017)	34.7	55.5	36.7	13.5	38.1	52.0	-	-	-	-	-	-
Faster R-CNN+++ (He et al., 2016)	ResNet-101	34.9	55.7	37.4	15.6	38.7	50.9	-	-	-	-	-	-
Faster R-CNN w/ FPN (Lin et al., 2016)	ResNet-101	36.2	59.1	39.0	18.2	39.0	48.2	-	-	-	-	-	-
Faster R-CNN w/ TDM (Shrivastava et al., 2016)	Inception-ResNet-v2	36.8	57.7	39.2	16.2	39.8	52.1	31.6	49.3	51.9	28.1	56.6	71.1
D-FCN (Dai et al., 2017)	Aligned-Inception-ResNet	37.5	58.0	-	19.4	40.1	52.5	-	-	-	-	-	-
Regionlets (Xu et al., 2017)	ResNet-101	39.3	59.8	-	21.7	43.7	50.9	-	-	-	-	-	-
Mask R-CNN (He et al., 2017)	ResNeXt-101	39.8	62.3	43.4	22.1	43.2	51.2	-	-	-	-	-	-
Soft-NMS (Bodla et al., 2017)	Aligned-Inception-ResNet	40.9	62.8	-	23.3	43.6	53.3	-	-	-	-	-	-
LH R-CNN (Li et al., 2017)	ResNet-101	41.5	-	-	25.2	45.3	53.1	-	-	-	-	-	-
Fitness-NMS (Tychsen-Smith and Petersson, 2017b)	ResNet-101	41.8	60.9	44.9	21.5	45.0	57.5	-	-	-	-	-	-
Cascade R-CNN (Cai and Vasconcelos, 2017)	ResNet-101	42.8	62.1	46.3	23.7	45.5	55.2	-	-	-	-	-	-
D-RFCN + SNIP (Singh and Davis, 2017)	DPN-98 (Chen et al., 2017)	45.7	67.3	51.1	29.3	48.8	57.1	-	-	-	-	-	-
One-stage detectors													
YOLOv2 (Redmon and Farhadi, 2016)	DarkNet-19	21.6	44.0	19.2	5.0	22.4	35.5	20.7	31.6	33.3	9.8	36.5	54.4
DSOD300 (Shen et al., 2017a)	DS/64-192-48-1	29.3	47.3	30.6	9.4	31.5	47.0	27.3	40.7	43.0	16.7	47.1	65.0
GRP-DSOD320 (Shen et al., 2017b)	DS/64-192-48-1	30.0	47.9	31.8	10.9	33.6	46.3	28.0	42.1	44.5	18.8	49.1	65.0
SSD513 (Liu et al., 2016)	ResNet-101	31.2	50.4	33.3	10.2	34.5	49.8	28.3	42.1	44.4	17.6	49.2	65.8
DSSD513 (Fu et al., 2017)	ResNet-101	33.2	53.3	35.2	13.0	35.4	51.1	28.9	43.5	46.2	21.8	49.1	66.4
RefineDet512 (single scale) (Zhang et al., 2017)	ResNet-101	36.4	57.5	39.5	16.6	39.9	51.4	-	-	-	-	-	-
RetinaNet800 (Lin et al., 2017)	ResNet-101	39.1	59.1	42.3	21.8	42.7	50.2	-	-	-	-	-	-
RefineDet512 (multi scale) (Zhang et al., 2017)	ResNet-101	41.8	62.9	45.7	25.6	45.1	54.1	-	-	-	-	-	-
CornerNet511 (single scale)	Hourglass-104	40.6	56.4	43.2	19.1	42.8	54.3	35.3	54.7	59.4	37.4	62.4	77.2
CornerNet511 (multi scale)	Hourglass-104	42.2	57.8	45.2	20.7	44.8	56.6	36.6	55.9	60.3	39.5	63.2	77.3

Fig. 10 示例边界框预测叠加在预测的角点热图上。

4.4.5 Quality of the Bounding Boxes

一个好的检测器应能预测出紧密覆盖物体的高质量边界框。为评估CornerNet所预测边界框的质量，我们在多个IoU阈值下测试了CornerNet的性能，并将结果与其他前沿检测器进行对比，包括RetinaNet（Lin等人，2017）、Cascade R-CNN（Cai与Vasconcelos，2017）以及IoU-Net（Jiang等人，2018）。

表5显示，CornerNet在0.9 IoU下获得的AP远高于其他检测器，比Cascade R-CNN + IoU-Net高出3.9%，比Cascade R-CNN高出7.6%，比RetinaNet [2]高出7.3%。这表明CornerNet能够生成比其他最先进检测器质量更高的边界框。

4.4.6 Error Analysis

CornerNet同时输出热图、偏移量和嵌入向量，这些都会影响检测性能。若任一角点未被检测到，目标就会漏检；需要精确的偏移量才能生成紧密的边界框；错误的嵌入向量则会导致大量误检边界框。为理解各部分对最终误差的贡献，我们通过将预测的热图和偏移量替换为真实值，并在验证集上评估性能来进行误差分析。

表6显示，仅使用真实角点热图就将AP从38.4%提升至73.1%。

[2] We use the best model publicly available on `https://github.com/facebookresearch/Detectron/blob/master/MODEL_ZOO.md`

Fig. 11 Qualitative examples on MS COCO.

AP^s, AP^m and AP^l also increase by 42.3%, 40.7% and 30.0% respectively. If we replace the predicted offsets with the ground-truth offsets, the AP further increases by 13.0% to 86.1%. This suggests that although there is still ample room for improvement in both detecting and grouping corners, the main bottleneck is detecting corners. Fig. 9 shows some qualitative examples where the corner locations or embeddings are incorrect.

4.5 Comparisons with state-of-the-art detectors

We compare CornerNet with other state-of-the-art detectors on MS COCO test-dev (Tab. 7). With multi-scale evaluation, CornerNet achieves an AP of 42.2%, the state of the art among existing one-stage methods and competitive with two-stage methods.

5 Conclusion

We have presented CornerNet, a new approach to object detection that detects bounding boxes as pairs of corners. We evaluate CornerNet on MS COCO and demonstrate competitive results.

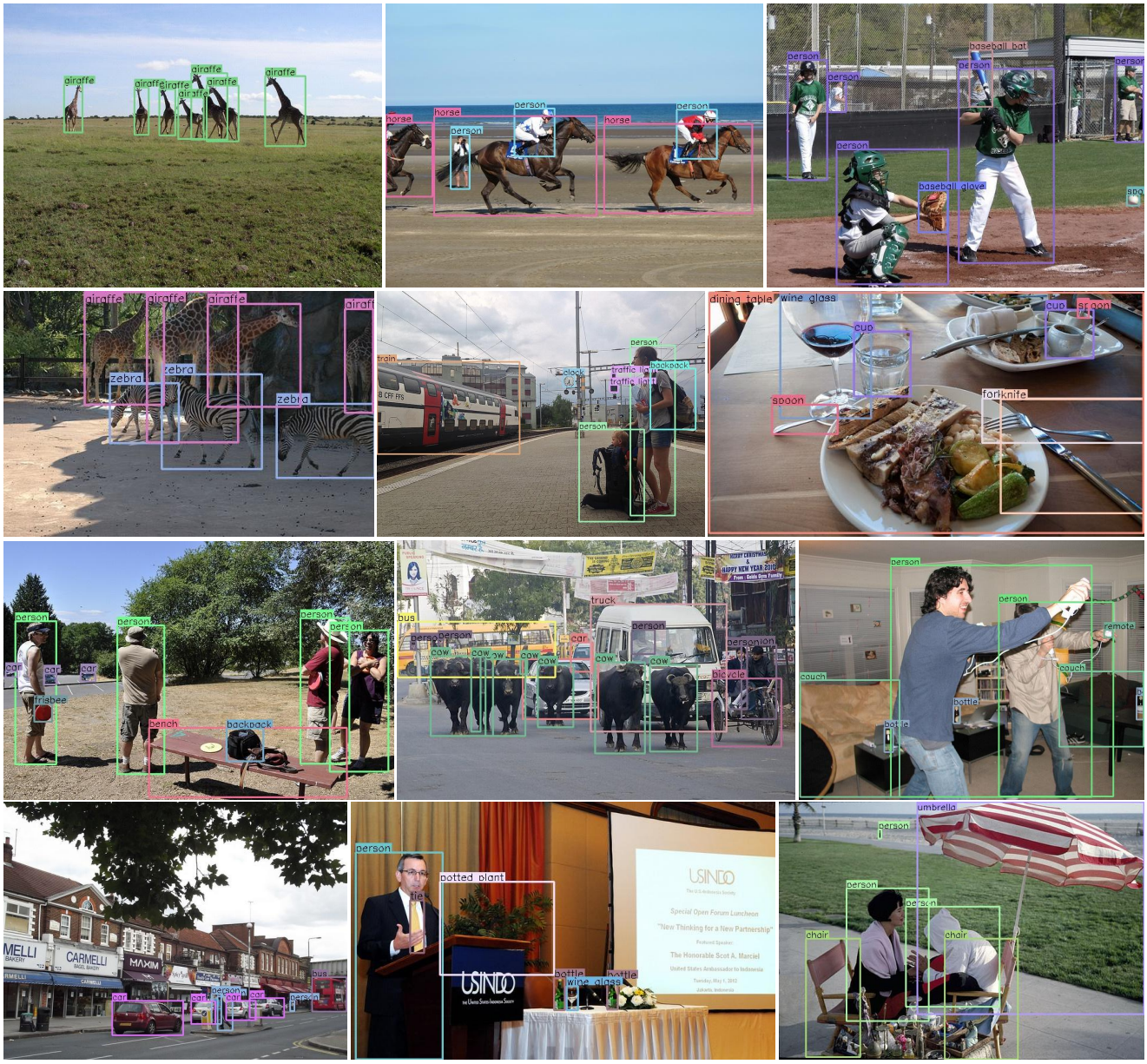


Fig. 11 MS COCO上的定性示例。

AP^s、AP^m和AP^l也分别增加了42.3%、40.7%和30.0%。若将预测偏移量替换为真实偏移量，AP会进一步提升13.0%至86.1%。这表明尽管在角点检测与分组方面仍有较大改进空间，但当前主要瓶颈在于角点检测环节。图9展示了一些角点位置或嵌入向量预测错误的定性示例。

4.5 与最先进检测器的比较

我们在MS COCO test-dev数据集上将CornerNet与其他先进检测器进行比较（表7）。通过多尺度在尺度评估中，CornerNet实现了42.2%的AP，在现有单阶段方法中达到领先水平，并与两阶段方法具有竞争力。

5 Conclusion

我们提出了CornerNet，一种新的目标检测方法，通过检测成对的角点来定位边界框。我们在MS COCO数据集上评估了CornerNet，并展示了具有竞争力的结果。

Acknowledgements This work is partially supported by a grant from Toyota Research Institute and a DARPA grant FA8750-18-2-0019. This article solely reflects the opinions and conclusions of its authors.

References

Bell, S., Lawrence Zitnick, C., Bala, K., and Girshick, R. (2016). Inside-outside net: Detecting objects in context with skip pooling and recurrent neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2874–2883.

Bodla, N., Singh, B., Chellappa, R., and Davis, L. S. (2017). Soft-nmsimproving object detection with one line of code. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 5562–5570. IEEE.

Cai, Z., Fan, Q., Feris, R. S., and Vasconcelos, N. (2016). A unified multi-scale deep convolutional neural network for fast object detection. In *European Conference on Computer Vision*, pages 354–370. Springer.

Cai, Z. and Vasconcelos, N. (2017). Cascade r-cnn: Delving into high quality object detection. *arXiv preprint arXiv:1712.00726*.

Chen, Y., Li, J., Xiao, H., Jin, X., Yan, S., and Feng, J. (2017). Dual path networks. In *Advances in Neural Information Processing Systems*, pages 4470–4478.

Dai, J., Li, Y., He, K., and Sun, J. (2016). R-fcn: Object detection via region-based fully convolutional networks. *arXiv preprint arXiv:1605.06409*.

Dai, J., Qi, H., Xiong, Y., Li, Y., Zhang, G., Hu, H., and Wei, Y. (2017). Deformable convolutional networks. *CoRR, abs/1703.06211*, 1(2):3.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*, pages 248–255. IEEE.

Everingham, M., Eslami, S. A., Van Gool, L., Williams, C. K., Winn, J., and Zisserman, A. (2015). The pascal visual object classes challenge: A retrospective. *International journal of computer vision*, 111(1):98–136.

Fu, C.-Y., Liu, W., Ranga, A., Tyagi, A., and Berg, A. C. (2017). Dssd: Deconvolutional single shot detector. *arXiv preprint arXiv:1701.06659*.

Girshick, R. (2015). Fast r-cnn. *arXiv preprint arXiv:1504.08083*.

Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587.

He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask r-cnn. arxiv preprint arxiv: 170306870.

He, K., Zhang, X., Ren, S., and Sun, J. (2014). Spatial pyramid pooling in deep convolutional networks for visual recognition. In *European Conference on Computer Vision*, pages 346–361. Springer.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.

Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., Fischer, I., Wojna, Z., Song, Y., Guadarrama, S., et al. (2017). Speed/accuracy trade-offs for modern convolutional object detectors. In *IEEE CVPR*.

Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456.

Jiang, B., Luo, R., Mao, J., Xiao, T., and Jiang, Y. (2018). Acquisition of localization confidence for accurate object detection. In *Computer Vision–ECCV 2018*, pages 816–832. Springer.

Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Kong, T., Sun, F., Yao, A., Liu, H., Lu, M., and Chen, Y. (2017). Ron: Reverse connection with objectness prior networks for object detection. *arXiv preprint arXiv:1707.01691*.

Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105.

Li, Z., Peng, C., Yu, G., Zhang, X., Deng, Y., and Sun, J. (2017). Light-head r-cnn: In defense of two-stage object detector. *arXiv preprint arXiv:1711.07264*.

Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2016). Feature pyramid networks for object detection. *arXiv preprint arXiv:1612.03144*.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. (2014). Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer.

Acknowledgements 本工作部分由丰田研究院的资助和DARPA资助FA8750-18-2-0019支持。本文仅代表作者的观点和结论。

References

Bell, S., Lawrence Zitnick, C., Bala, K., and Girshick, R. (2016). Inside-outside net: 通过跳跃池化和循环神经网络在上下文中检测物体。载于 *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 第2874–2883页。 Bodla, N., Singh, B., Chellappa, R., and Davis, L. S. (2017). Soft-nms：用一行代码改进物体检测。载于 *2017 IEEE International Conference on Computer Vision (ICCV)*, 第5562–5570页。 IEEE。 Cai, Z., Fan, Q., Feris, R. S., and Vasconcelos, N. (2016). 一种用于快速物体检测的统一多尺度深度卷积神经网络。载于 *European Conference on Computer Vision*, 第354–370页。 Springer。 Cai, Z. and Vasconcelos, N. (2017). Cascade r-cnn：深入高质量物体检测。*arXiv preprint arXiv:1712.00726*。 Chen, Y., Li, J., Xiao, H., Jin, X., Yan, S., and Feng, J. (2017). 双路径网络。载于 *Advances in Neural Information Processing Systems*, 第4470–4478页。 Dai, J., Li, Y., He, K., and Sun, J. (2016). R-fcn：通过基于区域的全卷积网络进行物体检测。*arXiv preprint arXiv:1605.06409*。 Dai, J., Qi, H., Xiong, Y., Li, Y., Zhang, G., Hu, H., and Wei, Y. (2017). 可变形卷积网络。*CoRR, abs/1703.06211*, 1(2):3。 Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet：一个大规模分层图像数据库。载于 *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*, 第248–255页。 IEEE。 Everingham, M., Eslami, S. A., Van Gool, L., Williams, C. K., Winn, J., and Zisserman, A. (2015). PASCAL视觉物体类别挑战：回顾。*International journal of computer vision*, 111(1):98–136。 Fu, C.-Y., Liu, W., Ranga, A., Tyagi, A., and Berg, A. C. (2017). DSSD：反卷积单次检测器。*arXiv preprint arXiv:1701.06659*。 Girshick, R. (2015). Fast r-cnn。*arXiv preprint arXiv:1504.08083*。 Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). 用于精确物体检测和语义分割的丰富特征层次结构。载于 *Proceedings of the IEEE conference on computer vision and pattern recognition*第580–587页。 He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask R-CNN. arXiv preprint arXiv:1703.06870. He, K., Zhang, X., Ren, S., and Sun, J. (2014). 用于视觉识别的深度卷积网络中的空间金字塔池化。载于 *European Conference on Computer Vision*, 第346–361页。 Springer。 He, K., Zhang, X., Ren, S., and Sun, J. (2016). 用于图像识别的深度残差学习。载于 *Proceedings of the IEEE conference on computer vision and pattern recognition*, 第770–778页。 Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., Fischer, I., Wojna, Z., Song, Y., Guadarrama, S., 等人 (2017). 现代卷积目标检测器的速度/精度权衡。载于 *IEEE CVPR*。 Ioffe, S. and Szegedy, C. (2015). 批量归一化：通过减少内部协变量偏移加速深度网络训练。载于 *International conference on machine learning*, 第448–456页。 Jiang, B., Luo, R., Mao, J., Xiao, T., and Jiang, Y. (2018). 获取定位置信度以实现精确目标检测。载于 *Computer Vision– ECCV 2018*, 第816–832页。 Springer。 Kingma, D. P. and Ba, J. (2014). Adam：一种随机优化方法。*arXiv preprint arXiv:1412.6980*。 Kong, T., Sun, F., Yao, A., Liu, H., Lu, M., and Chen, Y. (2017). RON：结合目标性先验的反向连接网络用于目标检测。*arXiv preprint arXiv:1707.01691*。 Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). 使用深度卷积神经网络进行ImageNet分类。载于 *Advances in neural information processing systems*, 第1097–1105页。 Li, Z., Peng, C., Yu, G., Zhang, X., Deng, Y., and Sun, J. (2017). Light-Head R-CNN：为两阶段目标检测器辩护。*arXiv preprint arXiv:1711.07264*。 Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2016). 用于目标检测的特征金字塔网络。*arXiv preprint arXiv:1612.03144*。 Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). 用于密集目标检测的焦点损失。*arXiv preprint arXiv:1708.02002*。 Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. (2014). Microsoft COCO：上下文中的常见物体。载于 *European conference on computer vision*, 第740–755页。 Springer。

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., and Berg, A. C. (2016). Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer.

Newell, A. and Deng, J. (2017). Pixels to graphs by associative embedding. In *Advances in Neural Information Processing Systems*, pages 2168–2177.

Newell, A., Huang, Z., and Deng, J. (2017). Associative embedding: End-to-end learning for joint detection and grouping. In *Advances in Neural Information Processing Systems*, pages 2274–2284.

Newell, A., Yang, K., and Deng, J. (2016). Stacked hourglass networks for human pose estimation. In *European Conference on Computer Vision*, pages 483–499. Springer.

Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., and Lerer, A. (2017). Automatic differentiation in pytorch.

Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788.

Redmon, J. and Farhadi, A. (2016). Yolo9000: better, faster, stronger. *arXiv preprint*, 1612.

Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*, pages 91–99.

Shen, Z., Liu, Z., Li, J., Jiang, Y.-G., Chen, Y., and Xue, X. (2017a). Dsod: Learning deeply supervised object detectors from scratch. In *The IEEE International Conference on Computer Vision (ICCV)*, volume 3, page 7.

Shen, Z., Shi, H., Feris, R., Cao, L., Yan, S., Liu, D., Wang, X., Xue, X., and Huang, T. S. (2017b). Learning object detectors from scratch with gated recurrent feature pyramids. *arXiv preprint arXiv:1712.00886*.

Shrivastava, A., Sukthankar, R., Malik, J., and Gupta, A. (2016). Beyond skip connections: Top-down modulation for object detection. *arXiv preprint arXiv:1612.06851*.

Simonyan, K. and Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.

Singh, B. and Davis, L. S. (2017). An analysis of scale invariance in object detection-snip. *arXiv preprint arXiv:1711.08189*.

Szegedy, C., Ioffe, S., Vanhoucke, V., and Alemi, A. A. (2017). Inception-v4, inception-resnet and the impact of residual connections on learning. In *AAAI*, volume 4, page 12.

Tychsen-Smith, L. and Petersson, L. (2017a). Denet: Scalable real-time object detection with directed sparse sampling. *arXiv preprint arXiv:1703.10295*.

Tychsen-Smith, L. and Petersson, L. (2017b). Improving object localization with fitness nms and bounded iou loss. *arXiv preprint arXiv:1711.00164*.

Uijlings, J. R., van de Sande, K. E., Gevers, T., and Smeulders, A. W. (2013). Selective search for object recognition. *International journal of computer vision*, 104(2):154–171.

Wang, X., Chen, K., Huang, Z., Yao, C., and Liu, W. (2017). Point linking network for object detection. *arXiv preprint arXiv:1706.03646*.

Xiang, Y., Choi, W., Lin, Y., and Savarese, S. (2016). Subcategory-aware convolutional neural networks for object proposals and detection. *arXiv preprint arXiv:1604.04693*.

Xu, H., Lv, X., Wang, X., Ren, Z., and Chellappa, R. (2017). Deep regionlets for object detection. *arXiv preprint arXiv:1712.02408*.

Zhai, Y., Fu, J., Lu, Y., and Li, H. (2017). Feature selective networks for object detection. *arXiv preprint arXiv:1711.08879*.

Zhang, S., Wen, L., Bian, X., Lei, Z., and Li, S. Z. (2017). Single-shot refinement neural network for object detection. *arXiv preprint arXiv:1711.06897*.

Zhu, Y., Zhao, C., Wang, J., Zhao, X., Wu, Y., and Lu, H. (2017). Couplenet: Coupling global structure with local parts for object detection. In *Proc. of Intl Conf. on Computer Vision (ICCV)*.

Zitnick, C. L. and Dollár, P. (2014). Edge boxes: Locating object proposals from edges. In *European Conference on Computer Vision*, pages 391–405. Springer.

刘、Anguelov、Erhan、Szegedy、Reed、傅、Berg（2016）。SSD：单次多框检测器。载于 *European conference oncomputer vision*，第21–37页。Springer出版社。Newell与Deng（2017）。通过关联嵌入从像素到图。载于*Advances in Neural Information Processing Systems*，第2168–2177页。Newell、Huang与Deng（2017）。关联嵌入：用于联合检测与分组的端到端学习。载于 *Advances in Neural InformationProcessing Systems*，第2274–2284页。Newell、Yang与Deng（2016）。用于人体姿态估计的堆叠沙漏网络。载于 *European Conference on Computer Vision*，第483–499页。Springer出版社。Paszke、Gross、Chintala、Chanan、Yang、DeVito、Lin、Desmaison、Antiga与Lerer（2017）。PyTorch中的自动微分。Redmon、Divvala、Girshick与Farhadi（2016）。你只看一次：统一、实时的目标检测。载于*Proceedings of the IEEE conference on computer vision and pattern recognition*，第779–788页。Redmon与Farhadi（2016）。YOLO9000：更好、更快、更强。*arXiv preprint*，1612。任、何、Girshick与孙（2015）。Faster R-CNN：基于区域提议网络实现实时目标检测。载于*Advances in neural information processing systems*，第91–99页。沈、刘、李、姜、陈与薛（2017a）。DSOD：从零开始学习深度监督目标检测器。载于*The IEEE International Conference on Computer Vision (ICCV)*，第3卷，第7页。沈、石、Feris、曹、闫、刘、王、薛与黄（2017b）。通过门控循环特征金字塔从零开始学习目标检测器。*arXivpreprint arXiv:1712.00886*。Shrivastava、Sukthankar、Malik与Gupta（2016）。超越跳跃连接：用于目标检测的自顶向下调制。*arXiv preprint arXiv:1612.06851*。Simonyan与Zisserman（2014）。用于大规模图像识别的极深卷积网络。*arXiv preprint arXiv:1409.1556*。Singh与Davis（2017）。目标检测中尺度不变性分析-SNIP。*arXiv preprint arXiv:1711.08189*。Szegedy、Ioffe、Vanhoucke与Alemi（2017）。Inception-v4、Inception-ResNet及残差连接对学习的影响。载于*AAAI*，第4卷，第12页。Tychsen-Smith, L. 与 Petersson, L. (2017a)。Denet：基于定向稀疏采样的可扩展实时目标检测。*arXiv preprint arXiv:1703.10295*。Tychsen-Smith, L. 与 Petersson, L. (2017b)。通过适应度非极大值抑制与有界交并比损失改进目标定位。*arXiv preprint arXiv:1711.00164*。Uijlings, J. R., van de Sande, K. E., Gevers, T., 与 Smeulders, A. W. (2013)。用于目标识别的选择性搜索。*International journal of computer vision*, 104(2):154–171。Wang, X., Chen, K., Huang, Z., Yao, C., 与 Liu, W. (2017)。用于目标检测的点链接网络。*arXiv preprint arXiv:1706.03646*。Xiang, Y., Choi, W., Lin, Y., 与 Savarese, S. (2016)。用于目标提议与检测的子类别感知卷积神经网络。*arXiv preprint arXiv:1604.04693*。Xu, H., Lv, X., Wang, X., Ren, Z., 与 Chellappa, R. (2017)。用于目标检测的深度区域网络。*arXiv preprint arXiv:1712.02408*。Zhai, Y., Fu, J., Lu, Y., 与 Li, H. (2017)。用于目标检测的特征选择网络。*arXiv preprint arXiv:1711.08879*。Zhang, S., Wen, L., Bian, X., Lei, Z., 与 Li, S. Z. (2017)。用于目标检测的单次优化神经网络。*arXiv preprint arXiv:1711.06897*。Zhu, Y., Zhao, C., Wang, J., Zhao, X., Wu, Y., 与 Lu, H. (2017)。Couplenet：将全局结构与局部部件耦合用于目标检测。收录于 *Proc. of Intl Conf. on Computer Vision (ICCV)*。Zitnick, C. L. 与 Dollár, P. (2014)。边缘框：从边缘定位目标提议。收录于 *European Conference on Computer Vision*, 第391–405页。Springer。